

分类号\_\_\_\_\_

学校代码 10487

学号 M201773200

密级\_\_\_\_\_

# 华中科技大学

# 硕士学位论文

## 基于 DPDK 的防火墙的设计与实现

学位申请人 邹婉晨

学科专业： 计算机技术

指导教师： 梅松 讲师

答辩日期： 2019.5.28

**A Thesis Submitted in Partial Fulfillment of the Requirements  
for the Degree for the Master of Engineering**

**Design and Implementation of Firewall  
Based on DPDK**

**Candidate : Zou Wanchen**

**Major : Computer Technology**

**Supervisor : Lec. Mei Song**

**Huazhong University of Science & Technology**

**Wuhan 430074, P.R.China**

**May, 2019**



## 摘 要

随着光纤技术的发展,网络带宽快速提升,硬件性能极速提高,但软件技术相对落后,基于传统的网络协议栈的防火墙对数据包的处理流程复杂,开销较大,已无法满足人们对于高性能处理密集数据的要求。

为了应对通用服务器内核网络处理功能低效的问题,研究了数据面开发工具集(Data Plane Development Kit,DPDK),和基于 DPDK 开发的用户态协议栈 f-stack,设计实现了基于 DPDK 的高性能防火墙。主要工作和创新点在以下几个方面:

利用 DPDK 的轮询技术拦截中断,解决中断处理带来的损耗问题,并利用 UIO(Userspace I/O)技术绕过内核协议栈,通过 UIO 技术将网卡收到的报文映射到用户态协议栈 f-stack 处理的工作原理,大幅减少捕获数据包时的性能消耗,提高防火墙处理数据包的性能。

在数据包进入 f-stack 协议栈之前增加了一条快速转发路径,对 DPDK 捕获到的数据包进行会话检查,对于已经建立会话的流,直接从记下的端口转发出去,以降低数据包在经过协议栈的过程中造成的性能损失,提高包转发速率。

优化了规则匹配算法,利用基于分治法的规则查询算法进行规则匹配,根据协议类型将规则集划分多个子规则集,减少规则匹配是规则集数量,并根据规则间的关系对规则进行分组,由组内的规则特征选择不同的查询算法,有效提高数据包在子规则集中的查询效率,同时也提高了规则查询模块整体的处理性能。

最后对防火墙进行功能和性能测试,验证了防火墙的包过滤功能和快速转发功能生效;在高速网络环境下具有较低的丢包率,防火墙整体性能达到较高水平。

**关键词:** 数据面开发工具集; 用户态协议栈; 防火墙; 快速转发; 规则匹配

## Abstract

In a complex network environment, a firewall is a barrier for network security. Configuring a firewall is one of the most common, economical, and effective security measures for network security. However, with the development of optical fiber technology, the network bandwidth is rapidly improved, and the hardware performance is extremely improved. By contrast, the software technology is relatively backward. The traditional network protocol stack has a complicated processing procedure and cause too much loss of performance, which cannot meet the requirements of processing massive data in a high performance way. This article is dedicated to improving the performance of the firewall system, such as throughput and packet forwarding rate.

In order to deal with the problem of inefficiency processing way of the general server kernel network, this paper studies the Data Plane Development Kit (DPDK) and the user-mode protocol stack f-stack based on DPDK. And designed a high performance firewall based on dpdk to processes packets efficiently. The main work and innovations are as follows:

This system uses the DPDK to intercept the interrupt, does not trigger the subsequent interrupt process, and bypasses the kernel protocol stack. The system uses the UIO technology mapping the packets received by the NICs to the user-mode protocol stack f-stack to process, greatly reducing the time when the data packet is captured. The performance consumption increases the performance of the firewall processing packets and reduces the system performance loss caused by the firewall.

The system adds a fast forwarding path before the data packet enters the f-stack protocol stack, performs session check on the data packet captured by the DPDK, and forwards the flow of the established session directly from the recorded port to reduce the

performance loss caused by the process of the protocol stack increases the packet forwarding rate.

The system optimizes the rule matching algorithm, uses the rule query algorithm based on divide and conquer method to perform rule matching, divides the rule set into multiple sub-rule sets according to the protocol type, reduces the number of rules to match, and divided the rules into different groups according to the relationship between the rules. Different query algorithms are selected by the rule features in the group, which effectively improves the query efficiency of the data packet in the sub-rule set, and also improves the overall processing performance of the rule query module.

Finally, the system performs functional and performance tests to verify that the packet filtering function and fast forwarding function of the firewall are effective. In the high-speed network environment, the packet loss rate is low, and the overall performance of the firewall reaches a high level.

**Keywords:** DPDK; f-stack; firewall; fast forwarding; rule matching

# 华中科技大学硕士学位论文

---

## 目 录

|                           |      |
|---------------------------|------|
| 摘 要.....                  | I    |
| Abstract .....            | II   |
| <b>1 绪论</b>               |      |
| 1.1 研究背景与意义 .....         | (1)  |
| 1.2 国内外研究概况 .....         | (2)  |
| 1.3 论文的主要研究内容 .....       | (5)  |
| 1.4 论文组织结构.....           | (6)  |
| <b>2 关键技术研究</b>           |      |
| 2.1 dpdk 技术简介.....        | (8)  |
| 2.2 用户态协议栈 f-stack.....   | (11) |
| 2.3 包过滤防火墙.....           | (13) |
| 2.4 本章小结.....             | (17) |
| <b>3 基于 DPDK 的防火墙方案设计</b> |      |
| 3.1 需求分析.....             | (19) |
| 3.2 总体设计.....             | (20) |
| 3.3 详细设计.....             | (22) |
| 3.4 本章小结.....             | (29) |
| <b>4 基于 DPDK 的防火墙实现</b>   |      |
| 4.1 数据包捕获模块实现 .....       | (31) |
| 4.2 会话查询模块实现 .....        | (32) |
| 4.3 规则查询模块实现 .....        | (35) |
| 4.4 快转模块实现.....           | (41) |

# 华中科技大学硕士学位论文

---

|                         |      |
|-------------------------|------|
| 4.5 管理模块实现.....         | (42) |
| 4.6 本章小结.....           | (45) |
| <b>5 基于 DPDK 的防火墙测试</b> |      |
| 5.1 功能测试.....           | (46) |
| 5.2 性能测试.....           | (53) |
| 5.3 本章小结.....           | (61) |
| <b>6 总结与展望</b>          |      |
| 6.1 全文总结.....           | (62) |
| 6.2 课题展望.....           | (63) |
| 致谢.....                 | (65) |
| 参考文献.....               | (66) |



## 1 绪论

随着网络技术的不断发展,现实生活中的各种组织系统都与网络息息相关<sup>[1]</sup>,网络给现代社会生活带来越来越多的便利,但是与此同时网络安全问题<sup>[2]</sup>也愈发突出和棘手。为使网络达到一定的安全性,许多网络安全技术应运而生,其中防火墙的使用范围最为广泛。网络用户利用防火墙作为内外网隔离的屏障<sup>[3]</sup>,使其根据一系列的访问控制规则实施安全策略,防止外网用户未的非法访问,保护了内网的安全<sup>[4]</sup>。网络防火墙发展至今,安全技术趋于成熟,着力于性能方面研究高性能<sup>[5]</sup>的防火墙成为构建网络安全的一个重要课题。

本论文研究了基于 DPDK (Data Plane Development Kit, 数据面开发工具集)<sup>[6]</sup>的高性能防火墙的实现技术,对能够实现高性能的 DPDK 和保证安全性的防火墙进行了深入研究,实现了二者结合的高性能防火墙。

### 1.1 研究背景与意义

随着互联网技术的发展,网络环境<sup>[7]</sup>变得越来越复杂,网络安全问题也层出不穷,日益突出。为了应对这些安全问题<sup>[8]</sup>,网络安全技术逐渐发展起来。其中防火墙是维护系统安全的第一道防线,被设置在内网与外网、专用网与公网之间<sup>[9]</sup>。它按照管理员在系统内部设定的规则来控制数据包的进出,其作用是防止非法用户的进入。

然而随着光纤技术的发展<sup>[10]</sup>,网络带宽快速提升,硬件性能极速提高,但软件技术相对落后,传统的网络协议栈对数据包的处理流程复杂,开销较大,基于传统网络协议栈的防火墙已无法满足人们对于高性能处理密集数据的要求,因此研究开发高性能防火墙具有十分重要的意义。

为了提高协议栈处理数据包的性能,研究人员将目光放在了用户态协议栈<sup>[11]</sup>,用户态协议栈提升性能的原因在于两个方面:第一,用户态协议栈能够对协议栈进行

裁剪，仅保留需要的协议，定制性强，方便根据实际情况进行性能优化；第二，避免了内核态与用户态之间数据交互的拷贝和系统调用。

本文的目的在于研究并开发出一个将用户态协议栈和防火墙结合的高性能防火墙。

## 1.2 国内外研究概况

### 1.2.1 包过滤性能优化概况

数据包过滤技术已经相当成熟，包过滤防火墙通过检查每一个包的包头信息，根据规则判断对包进行接收、转发或丢弃。包过滤技术检查数据包包头，检查的主要信息有：源/目的 IP 地址、协议类型、TCP/UDP 包的源/目的端口号、ICMP 消息类型、TCP 包的 ACK 位、数据包到达/发出端口等。

国内外在包过滤防火墙的研究中除了对防火墙安全性的研究外，越来越多的人注意到对防火墙性能的优化。

从硬件方面的优化例如基于 FPGA (Field-Programmable Gate Array, 现场可编程门阵列)<sup>[12]</sup>开发的防火墙，调用对应的硬件模块来实现特定功能。2013 年印度尼西亚的万隆理工学院提出一种基于 FPGA 包分类机制用于实现快速过滤数据包，处理器采用 NIOS II 软核，可以实时更新包分类机制，实现灵活配置。

从软件方面的优化包括规则集<sup>[13]</sup>的优化、对操作系统的优化<sup>[14]</sup>和对协议栈<sup>[15]</sup>的优化。最近几年国内外各大高校和研究机构在用户态协议栈的相关技术的研究上取得了许多成果。华盛顿大学设计实现了一个基于 FreeBSD 协议栈的用户态框架 Alpine，它将内核空间的服务虚拟化为一个用户空间的协议栈，并且能够共享内核状态，减小了收发数据包的周期<sup>[16]</sup>。在国内，清华大学设计了一个用户空间的通信协议 RCP，在数据传输路径上，RCP 绕过操作系统内核，使网络设备和用户空间直接互通，减少网络数据包传递到用户空间的拷贝次数，提高了网络吞吐量<sup>[17]</sup>。

## 1.2.2 网络数据包处理性能优化概况

近些年来,随着网络带宽的不断提升,网络数据包处理对系统的软硬件的性能需求越来越大,传统的通用架构的服务器已经无法适应高速流量的处理需求,因此市场上涌现出很多专用的网络流量处理服务器。但专用服务器价格昂贵,而且由于实际应用场景的多样化,专用服务器无法满足用户需求。在不断对两种服务器优化的研究中发现,通用性和性能成反比<sup>[18]</sup>,即性能越高,通用性越弱,而通用性越高,性能会减弱。例如目前通用的 TCP/IP 协议栈处理流量的性能与硬件相比要低<sup>[19]</sup>,但是协议栈的通用性优于硬件设备。

为了提高通用服务器的性能,改善其网络处理性能的瓶颈问题<sup>[20-23]</sup>,有很多研究提出了设计高性能软件数据包捕获机制。目前提出的较为常见的技术有 DPDK<sup>[24]</sup>、netmap<sup>[25]</sup>、PF\_RING ZC<sup>[26]</sup>等。

### (1) DPDK 的研究概况

2014 年 Adam Dunkels 基于 DPDK 实现了 LwIP 协议栈<sup>[19]</sup>,这是对 TCP/IP 协议栈精简后的轻量级协议栈,主要是通过减少内存使用和简化 TCP/IP 协议栈来提高网络数据包处理性能。Aravind C Ajayan、P Prabakaran 等人基于 DPDK 实现了 Hiper-Ping<sup>[27]</sup>,这是一种高性能数据包生成系统,利用 DPDK 高效处理数据包的机制在 mbuf 中生成数据包。Zhijun Xu、Liang Zhou 等人基于 DPDK 实现的 Packet Usher<sup>[28]</sup>这是一个基于 DPDK 的高性能数据包 I/O 引擎<sup>[5]</sup>,能够显著提高商用 PC 上的 I/O 密集型和计算密集型的应用程序的处理性能。加州大学河滨分校和华盛顿大学基于 DPDK 和 Docker 容器实现了一种高性能 NFV (Network Function Virtualization, 网络功能虚拟化)平台<sup>[29]</sup>,可以启用 SDN(Software Defined Network, 软件定义网络),允许网络控制器提供规则,规定处理每个数据流所需的网络功能。何佳伟、方舟提出了一种基于 DPDK 的高性能网络安全审计方案<sup>[30]</sup>,该方案不仅具有良好的稳定性,而且在性能方面与传统的同类产品相比也占有优势。

### (2) Netmap 的研究概况

H.Redžović、M.Vesović<sup>[31]</sup>等人提出并评估了基于 netmap 平台的节能网络处理算法 - 节能网络映射,通过最小化处理给定流量负载所需的活动核心数来节省能量。

## (3) PF\_RING

Jin-Hong Kim、Jung-Chan Na<sup>[32]</sup>提出一种使用 PF\_RING ZC 进行单向通信的方案,通过寄存器修改单向链路设置和 PF\_RING ZC 库来提高性能。

## (4) 其他框架的研究

Sangjin Han、Keon Jang 等人在 2010 提出了使用图形处理器加速的高性能软件路由器框架 Packet Shader,达到了较高的数据包转发性能<sup>[33]</sup>,并提供了数据包处理接口,可以将其应用到多种数据包处理应用中。2014 年,Michio 等人实现了一种用户态 TCP 协议栈 Multi Stack、该协议栈拥有较好的扩展性,基于该协议栈开发应用程序,可以大大降低用户态应用程序开发难度<sup>[34]</sup>。Michio 等人基于 Multi Stack 开发了 HTTP 服务器,其 TCP 性能相比传统 Linux 内核协议栈高出 18%-19%。2015 年,TANG Lu 等人提出,使用 FPGA 可编程的专用网络处理器能达到较高的数据包处理性能<sup>[35]</sup>,但 Tang Lu 指出,专用网络处理器也有很多不足,例如,FPGA 高性能处理器造价高昂,无法使用通用架构的软件,开发难度较大,灵活性差,导致整体开发周期较长等。Tang Lu 等人提出了基于 x86 架构的通用处理器与 FPGA 结合使用的高速数据包处理机制,缩短了开发周期。2016 年,Li 等人提出一种基于 FPGA 的高性能报文处理加速平台,并提供 C 语言编程接口,吞吐量相较于传统内核协议栈有着几十倍的提高<sup>[36]</sup>,在 64~1500 字节的数据包包长情况下都能达到线速,单个数据包处理时延大大降低。

### 1.2.3 国内外现状解析

通过对国内外研究现状分析发现,通用架构服务器系统网络协议栈在进行高速报文处理时存在较大的性能瓶颈。针对网络协议栈处理低效的问题,研究人员开发了许多高速报文处理框架。这些报文处理框架分为两类,一类从硬件方面进行改进,例

如使用 GPU 或 FPGA 等辅助 CPU 进行数据包处理，一类从软件方面进行改进，以旁路方式进行数据包收发<sup>[37]</sup>，使用高性能数据包报文收发平台对网络数据包进行获取，再利用用户态协议栈加速报文处理。从硬件方面进行改进的框架需要借助于额外的硬件如 GPU 或 FPGA 芯片，难以移植，遇到问题也难以进行调试，在市场上占有率也不高。从软件方面改进的框架中，认可度较高，应用较为广泛的是 DPDK、netmap 和 PF\_RING ZC。在这三种框架中，DPDK 和 PF\_RING ZC 的性能高于 netmap。DPDK 采用 GPL 协议，完全免费开源，DPDK 的开发者众多，社区活跃，开发文档非常完善，而 PF\_RING ZC 使用 EULA License，要求为每个 MAC 地址申请一个许可证，限制较多。PF\_RING ZC 开发维护人员较少，在应用广泛程度上，PF\_RING ZC 也没有 DPDK 高，所以本文基于 DPDK 框架进行研究。

## 1.3 论文的主要研究内容

本文研究了 Linux 下基于 Netfilter 架构的防火墙运行原理，通过分析传统防火墙的机制，找出了对性能影响较大的三大因素是网络数据包处理、会话查询以及规则查询，通过对各个因素的详细研究，提出相应的改进方案。具体研究工作如下：

(1) 以高速网络环境下实现数据包快速转发和高效规则匹配为目的作需求分析，借鉴 f-stack 用户态协议栈的快速数据包处理架构，设计出一个符合需求的防火墙实现方案，进而提出本方案的整体架构和模块划分。

(2) 针对网络数据包处理性能进行优化，本文研究了 netmap、PF\_RING 和 DPDK 三种高性能报文处理框架，经过详细分析三者的优缺点，最后决定选择 DPDK 作为开发的基础框架，运用多核多线程技术，利用 CPU 亲和性，将 DPDK 的数据包捕获线程绑定到一个或多个 CPU 核上执行，使每个数据包捕获线程尽可能使用独立的资源进行数据包处理，减少 CPU 运行时的上下文切换和资源竞争，保证数据包捕获的性能。

(3) 针对会话查询性能进行优化，采用哈希算法对 IP 报文五元组做哈希处理，

通过分析多种哈希算法在高速网络环境下的性能，选择基于链地址法的哈希算法来保证会话查询的性能。

(4) 针对规则查询的算法进行性能优化，传统的规则匹配算法是顺序匹配算法，但是考虑到规则数量越来越多，顺序匹配算法的效率就会较为低下，因此用分组算法来优化规则查询的性能。分组算法将规则分为两组，如果某条规则与其他所有规则都是完全或部分无关，则属于 M 组；如果某条规则和其他规则相关，该规则和其相关的规则均属于 N 组。对两组规则匹配时针对两组的分组特征采用不同的查找算法，保证规则查询的性能。

## 1.4 论文组织结构

本文通过对基于传统协议栈防火墙的深入研究，分析出影响传统防火墙性能的原因，提出了一种基于高性能报文处理框架 DPDK 的防火墙方案，极大地提高防火墙处理数据包性能。本文共分为 6 章，按以下结构行文：

第一章，绪论。本章介绍了课题的研究背景与意义，分析了国内外在包过滤性能优化领域和网络数据包处理性能优化领域的现状，比较分析了 netmap、DPDK、PF\_RING 三种高性能报文处理框架的优缺点，结合实际的应用场景，本防火墙将基于 DPDK 框架开发。

第二章，关键技术研究。本章介绍了实现基于 DPDK 高性能防火墙所要用到的技术。包括 DPDK 技术简介，用户态协议栈 f-stack 的简介以及包过滤防火墙的两种框架介绍。DPDK 技术简介中介绍了 DPDK 的软件架构，以及 DPDK 为了提高数据包捕获性能所利用的巨页技术、轮询技术和 CPU 亲和技术。

第三章，基于 DPDK 的防火墙方案设计。本章首先提出了该方案的需求分析，然后给出方案的总体设计，最后给出各模块的详细设计，包括数据包捕获模块、会话查询模块、规则查询模块、快转路径模块、协议栈路径模块、日志模块和管理模块。

第四章，基于 DPDK 的防火墙实现。本章给出了各模块的具体实现方案，介绍

了各个模块的工作流程。

第五章，基于 DPDK 的防火墙测试。本章从功能和性能两个方面对本防火墙进行测试，重点介绍了性能方面的测试，通过对传统 Linux 协议栈转发性能、DPDK 转发性能、f-stack 三层转发性能、快转性能、防火墙整体性能进行测试，得到性能测试结果，得出结论。

第六章，总结与展望。分析全文内容，总结本防火墙方案的优缺点，并对此研究方向作出展望。

2 关键技术研究

本章将介绍要实现高性能防火墙所要用到的关键技术，包括有 DPDK 的软件架构及其使用的巨页技术、轮询技术和 CPU 亲和技术，基于 DPDK 开发的用戶态协议栈 f-stack 以及包过滤防火墙的两种典型架构：Netfilter/iptables 架构和 Pfil/ipfw 架构。

2.1 dpdk 技术简介

由 Intel、6WIND 等众多公司联合开发的 DPDK(Data Plane Development Kit)即数据面开发套件<sup>[37]</sup>，主要基于 Linux 操作系统运行，可以大幅提高数据包处理性能和吞吐量。

2.1.1 软件架构

DPDK 的组成架构如图 2-1 所示。

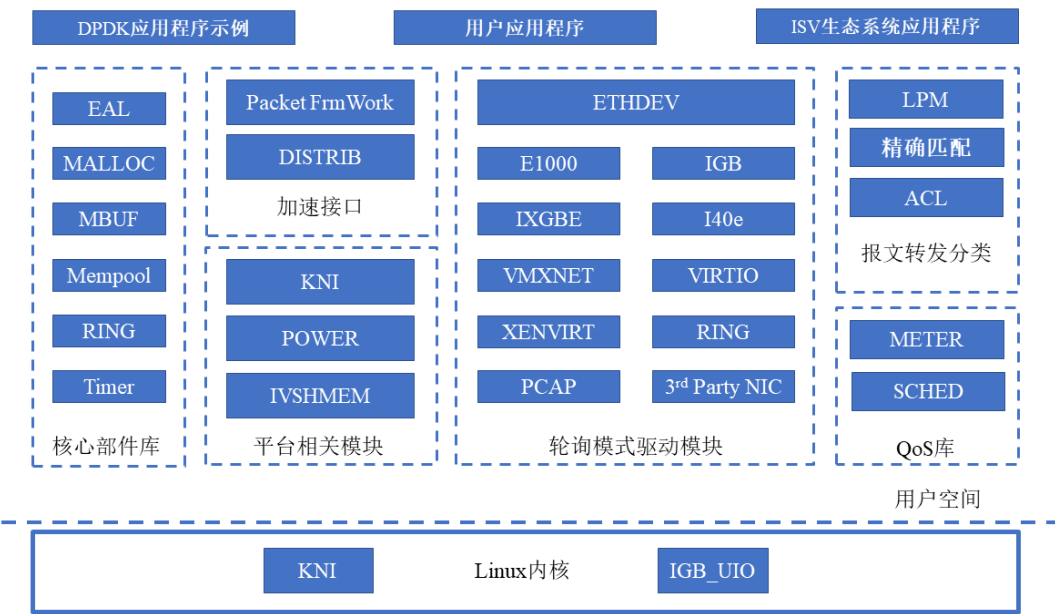


图 2-1 DPDK 组成架构

如图 2-1 所示，底部的内核态有 KNI(Kernel NIC Interface, 内核网卡接口)和



IGB\_UIO 两个模块, KNI 为 DPDK 应用程序提供与 Linux 内核通信的接口, IGB\_UIO 在 DPDK 初始化过程中将网卡收到的数据包从内核态映射到用户态<sup>[38]</sup>。

上层的用户态 DPDK 由众多的库<sup>[39]</sup>构成, 包括核心部件库、加速接口、平台相关模块、轮询模式驱动模块、报文转发分类、QoS 库等。

核心部件库基于 Linux 运行, 通过环境抽象层 (Environment Abstract Layer, EAL) 进行初始化, 包括分配大页内存、绑定进程到指定核上、分配队列等操作。利用 MALLOC 创建处理数据包的内存池, MBUF、Mempool、RING 作为内存管理的基本构成, Mempool 使用无锁队列结构 RING 存储数据载体 MBUF。Timer 起到定时器的作用, 通过 EAL 提供时间接口。

加速模块包括 Packet Framework 和 DISTRIB, 多核流水线处理模型的即是以这两者为基础构建的。

平台相关模型中的能耗管理(POWER)提供相关 API 供 DPDK 应用程序调整处理器状态, IVSHMEM 接口为虚拟机-虚拟机或主机-虚拟机间零拷贝内存共享提供优越的可行方案。

轮询模式驱动模块中的 API 实现了使用轮询模式代替中断模式收发数据包, 消减了由中断上下文切换造成的开销。轮询模式大幅提高了网卡处理数据包的性能。

报文转发分类模块中提供的转发算法包括最长前缀匹配算法(Longest Prefix Match, LPM)、精确匹配算法、通配符匹配算法等。

QoS 库提供差异化服务质量的相关库包括限速(METER)和调度(SCHED), 根据不同的用户或应用程序的服务需求提供不同质量的网络服务。

## 2.1.2 巨页技术

在 Linux 系统中, 对内存的管理包括两个方面, 物理内存<sup>[40]</sup>以帧为单位进行管理, 虚拟内存以页为单位进行管理。为了将虚拟内存转换为物理内存, 需要内存管理单元将内存地址转换信息保存到页表中, 通过查询页表完成地址转换。但是页表查询

耗时较多,对性能影响较大。在 Intel 处理器中用高速缓存(Translation Lookaside Buffer, TLB)保存虚拟内存到物理内存的映射关系,整个地址转换流程是 CPU 先在 TLB 中查找需要的映射关系是否存在,如果不存在(即 TLB miss)再进行页表查询。减少 TLB miss 是提高地址转换性能的关键。

x86 处理器的默认 TLB 指向大小为 4KB 的页表,也可以更改到更大例如 2MB 或 1GB。但巨页表功能可以使 TLB 指向更大的内存区域,极大地减少了 TLB miss,对性能提升有着巨大作用。

巨页是内核态的概念,而 DPDK 运行在用户态,为了能使用巨页技术,DPDK 需要使用 HUGETLBFS。先将大页内存挂载到某个路径下,DPDK 运行时会调用 mmap() 系统调用将大页内存映射到用户态地址空间后即可使用。

## 2.1.3 轮询技术

传统网卡硬件收发数据包后,向 CPU 发送中断请求,CPU 响应中断,进入中断处理函数处理相关事务。CPU 处理中断需要经过一系列的入栈出栈操作,整个过程需要消耗大量的时钟周期。可想而知 CPU 频繁进行中断处理会严重影响数据包收发的性能。

为了解决中断处理带来的损耗问题,DPDK 使用了轮询方式<sup>[41]</sup>收发数据包。在处理器缓存或内存头部设置一个标志位,记录数据包收发状态,数据包被发送后,该标志位将置为空闲状态,DPDK 应用程序的 polling 线程周期性地轮询该标志位,检测是否有数据包需要处理,若检测数据包发送成功将回收内存并回调。轮询方式中没有中断处理,使得数据包处理性能大幅提升。

## 2.1.4 CPU 亲和技术

尽管 DPDK 运行在用户态,但是其线程仍然由内核来调度,调度方式是基于分时调度。在多核处理器中,多个进程或线程在其中一个核中交替执行,需要进行频繁的进程或线程切换,每次切换 CPU 都要进行堆栈操作,这样的调度方式对系统来说

是一个很大的开销。

DPDK 利用 CPU 亲和技术<sup>[42]</sup>，将特定的进程或线程指定到某个核上工作来独占该核，避免了这些进程或线程的切换，减少了任务调度带来的开销，提升了专用程序的性能。

2.2 用户态协议栈 f-stack

由于 DPDK 不支持 TCP/IP 协议栈，对开发者来说是个不小的考验。目前的 DPDK 开发大多应用于 DNS、NFV 等简单应用场景，也开发了 mTCP、LwIP 等用户态协议栈以及用户态操作系统 Seastar，但是它们使用了 C++14、message-passing 等先进模式，对程序员来说上手门槛较高。

由腾讯开发的基于 DPDK 的开源高性能网络框架 f-stack 使用纯 C 实现，容易上手；将 FreeBSD TCP/IP 协议栈移植到用户态，功能完善运行稳定；POSIX API 基本兼容 socket/epoll/kqueue 等，便于原程序的接入和原系统的移植。基于这些原因本课题利用 f-stack 作为后续处理数据包的用户态协议栈。

2.2.1 总体架构

F-stack 是一个基于 DPDK 的高性能开源网络框架，如图 2-2 所示。

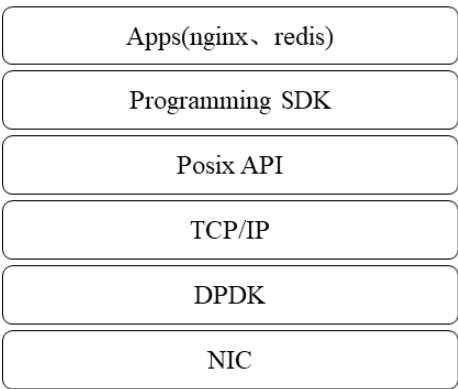


图 2-2 f-stack 总体架构

包括用户空间 TCP/IP 堆栈（基于 FreeBSD 11.0 稳定版），Posix API（Socket，

Epoll, Kqueue), Programming SDK (Coroutine) 和应用程序 (如 Nginx, Redis 等)。F-stack 使用多进程无共享架构, 如图 2-3 所示。每个进程绑定独立的 CPU 核和网卡队列, 通过网卡队列将数据流分发到不同的进程进行处理。由于跨 CPU 节点通信是非常耗时的, 因此, 在 f-stack 架构中, 每个 NUMA 核使用独立的内存池, 避免数据包在不同核心之间进行内存拷贝, 减少性能损耗。

F-stack 的每一个进程都单独运行在一个 CPU 核心上, 每个进程拥有单独的协议栈、进程控制块(Process Control Block, PCB)等资源, 同时, 进程间通过无锁环形队列进行进程通信, 可以有效避免死锁发生, 具有高性能、无竞争、零拷贝、线性扩展、NUMA(Non Uniform Memory Access Architecture, 非统一内存访问)友好、易用性高、通用性强等特点。

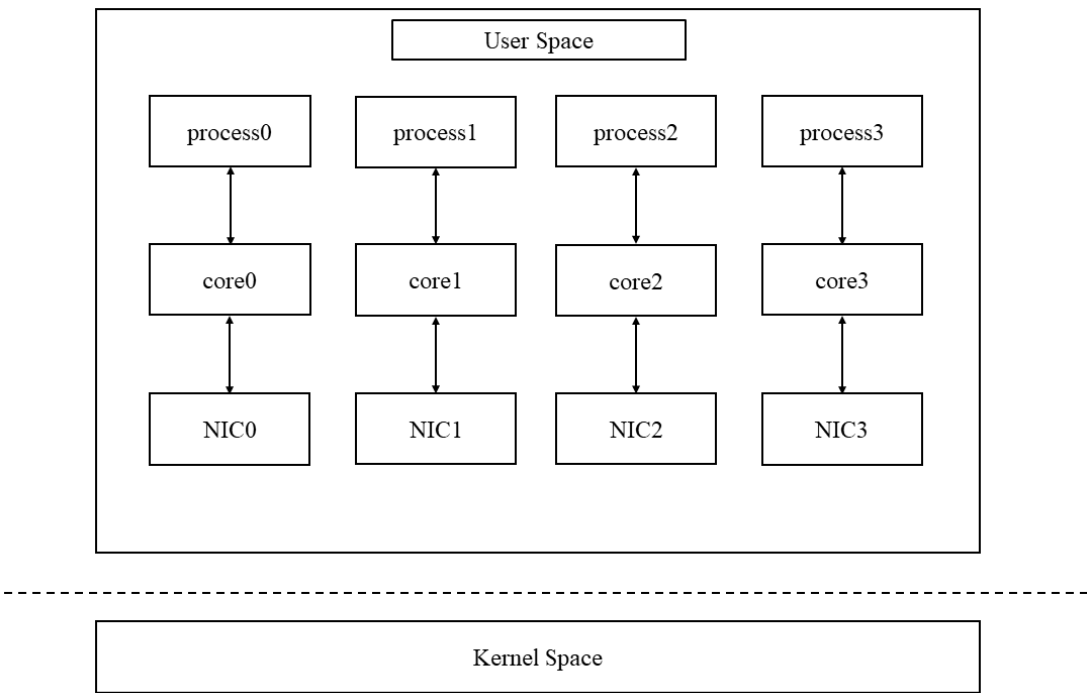


图 2-3 f-stack 无共享架构

### 2.2.2 基于 DPDK 的数据包处理能力

f-stack 在数据包处理方面的高性能还是依赖于 DPDK 的收发包机制, 例如用户

态驱动实现内存零拷贝、轮询机制等。F-stack 自身提升数据包处理性能的重要原因是采用了无共享的整体架构,每个进程或线程拥有独立的协议栈和表资源,实现了进程间资源无竞争。

2.2.3 基于 FreeBSD 的用户态协议栈

f-stack 的前身是腾讯开发的基于 DPDK 的授权 DNS 项目的 TCP 协议栈基础上实现的完整 TCP/IP 协议栈,但在线上运行近一年后,发现线上的网络环境非常复杂多变,这个协议栈不能满足所有的网络需求。经过调研后决定移植 FreeBSD 11.01 协议栈到用户态,削减了大量不相关的功能,并做了内存分配优化,移除内核锁等改进优化,大大提升了性能。

2.3 包过滤防火墙

包过滤防火墙是一种网络层防火墙,设置在内网和外网或专用网络和公网之间,根据过滤规则决定数据包是否通过。下文将介绍两种典型的包过滤防火墙架构:Netfilter/Iptables 架构和 Pfil/ipfw 架构。

2.3.1 Netfilter/iptables 架构简介

Netfilter 是内核态的通用框架,提供一套钩子(hook)函数管理机制。通过在网络数据包在协议栈中经过的关键点(HOOK 点)注册 hook 函数进行钩挂,每个 hook 函数将根据返回值的来决定数据包的流向,实现数据包过滤。下面以 ipv4 网络协议栈为例分析 Netfilter 框架。

在 ipv4 协议栈中有 5 个 HOOK 点,与每个 HOOK 点对应的功能如表 2-1 所示。

表 2-1 HOOK 点功能表

| HOOK 点            | 功能                |
|-------------------|-------------------|
| NF_IP_PRE_ROUTING | 目的地址转换            |
| NF_IP_LOCAL_IN    | 对目的地址是本机的数据包进行包过滤 |

|                    |                   |
|--------------------|-------------------|
| NF_IP_FORWRD       | 对目的地址不是本机的数据包进行转发 |
| NF_IP_LOCAL_OUT    | 对源地址是本机的数据包进行包过滤  |
| NF_IP_POST_ROUTING | 源地址转换             |

数据包在 ipv4 协议栈中的经过的 HOOK 节点如图 2-4 所示。

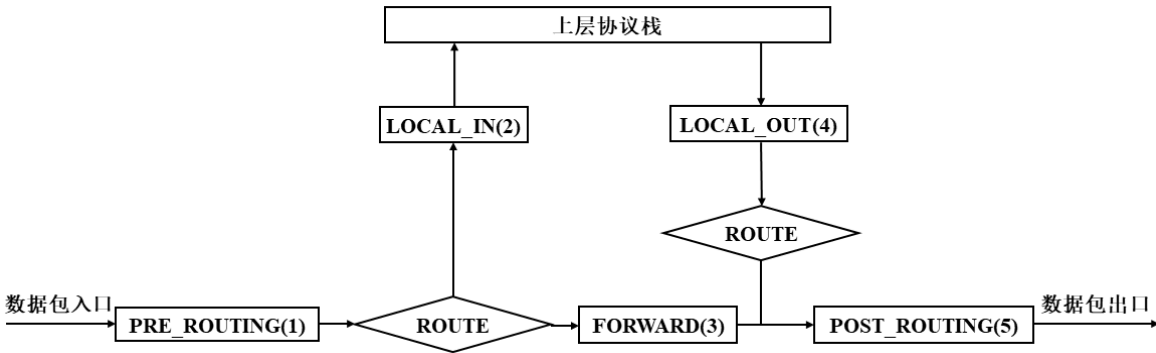


图 2-4 HOOK 节点

接收数据包时，数据包进入协议栈后，都要经过 PRE\_ROUTING 点，经过路由判决，如果目的地址是本机，则路由至 LOCAL\_IN 点，然后流向上层协议栈；如果目的地址不是本机，则路由至 FORWARD 点，然后经过 POST\_ROUTING 点将数据包转发出去。

发送数据包时，数据包先经过 LOCAL\_OUT 点，再经过路由判决流经 POST\_ROUTING 后将数据包发送出去。

每个 hook 函数有五种返回值，Netfilter 框架根据返回值决定数据包的处理结果。每种返回值的对应意义如表 2-2 所示。

表 2-2 hook 函数返回值

| 返回值       | 意义                                |
|-----------|-----------------------------------|
| NF_ACCEPT | 接收数据包                             |
| NF_DROP   | 丢弃数据包                             |
| NF_STOLEN | 数据包被 hook 函数处理但不进行后续 Netfilter 流程 |

|           |                    |
|-----------|--------------------|
| NF_QUEUE  | 数据包入队列并送往用户态处理     |
| NF_REPEAT | 再次调用该 hook 函数处理数据包 |

iptables 是用户态的规则编辑管理工具，规则集中的数据包控制规则是通过 Netfilter 机制实现的。Netfilter 与 iptables 的关系如图 2-5 所示。

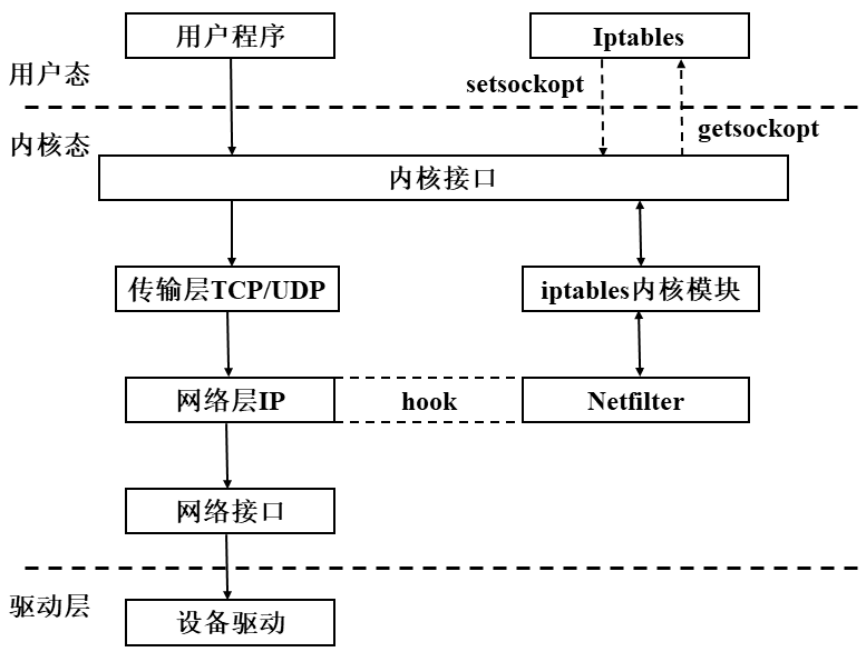


图 2-5 Netfilter-iptables 关系图

Iptables 提供了 4 张表，每个表对应的功能如表 2-3 所示。

表 2-3 iptables 表

| 表      | 功能     |
|--------|--------|
| Filter | 包过滤    |
| Nat    | 网络地址转换 |
| Mangle | 包重构    |
| Raw    | 数据包追踪  |

四种表的优先级是：raw>mangle>nat>filter。Netfilter 提供了 5 条链：PREROUTING 链、INPUT 链、FORWARD 链、OUTPUT 链和 POSTROUTING 链。

Iptables 表、链、规则的关系如图 2-6 所示。

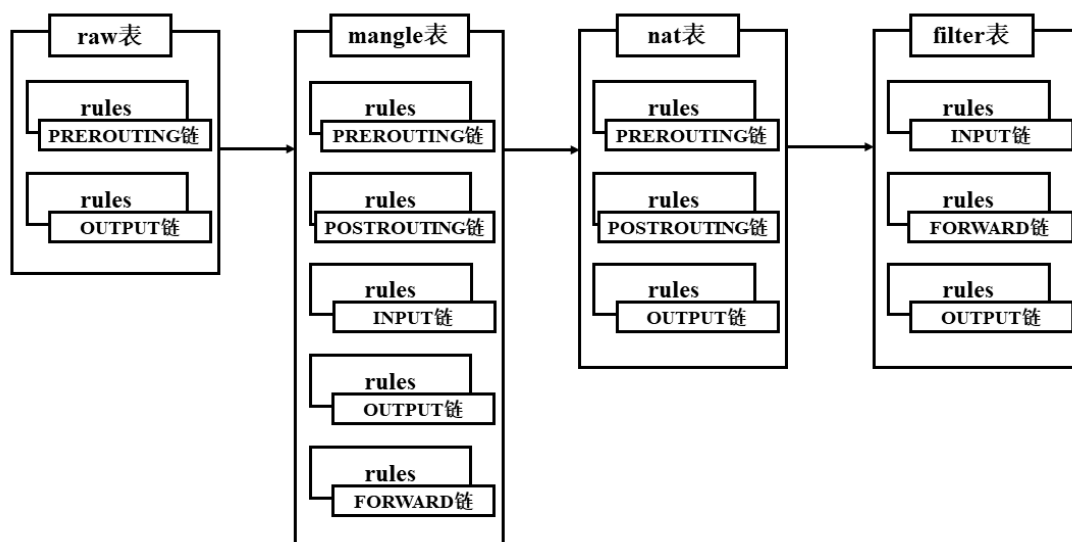


图 2-6 iptables 表、链、规则关系图

由图 2-6 可知，每个表有多条链，每条链中有多条有序规则，数据包按照规则顺序依次匹配，若匹配则跳过该链的后续规则。若匹配且 hook 函数返回 ACCEPT，数据包继续匹配后续链和表的规则；若匹配且 hook 函数返回 REJECT 或 DROP，数据包不再继续匹配。由于 netfilter/iptables 架构中的表、链、规则繁多，在有多数数据包尝试匹配的情况下，防火墙的包处理性能会大幅降低。

## 2.3.2 Pfil/ipfw 架构简介

FreeBSD 中也有类似于 Netfilter 的数据包处理机制 Pfil，Pfil 通过内核变量的作用对数据包在协议栈多处进行检查，控制数据包流向，实现数据包过滤。ipfw 的数据包处理流程如图 2-7 所示。

在数据链路层中注册网卡设备时，数据包从网卡传入 ether\_input 函数，根据 ether\_header 结构体判断数据包类型，根据类型将数据包分组，分别加入到对应的协议处理队列等待处理。若数据包是 ip 报文，则协议栈会将数据包传入 ether\_demux 函数进行网络层预处理，若数据包类型是 vlan 类型数据包，则 ether\_demux 会调用相



应的钩子函数进行解包处理,再传入网络层。`ip_input` 函数是网络层协议入口,若 `ipfw` 在 `ip` 层注册了钩子函数,则会先调用 `pfil_run_hooks` 遍历注册的钩子函数进行转发、丢弃、接受等处理。`ip_input` 会对数据包进行网络层拆包得到源目的 `ip` 地址,若目的地址是本机则传递给传输层和应用层进行处理。否则,协议栈将根据目的 `ip` 地址,查找路由,尝试进行转发,然后 `ip_output` 将数据包传入数据链路层 `ether_output` 函数,再由网卡设备将数据包发出。

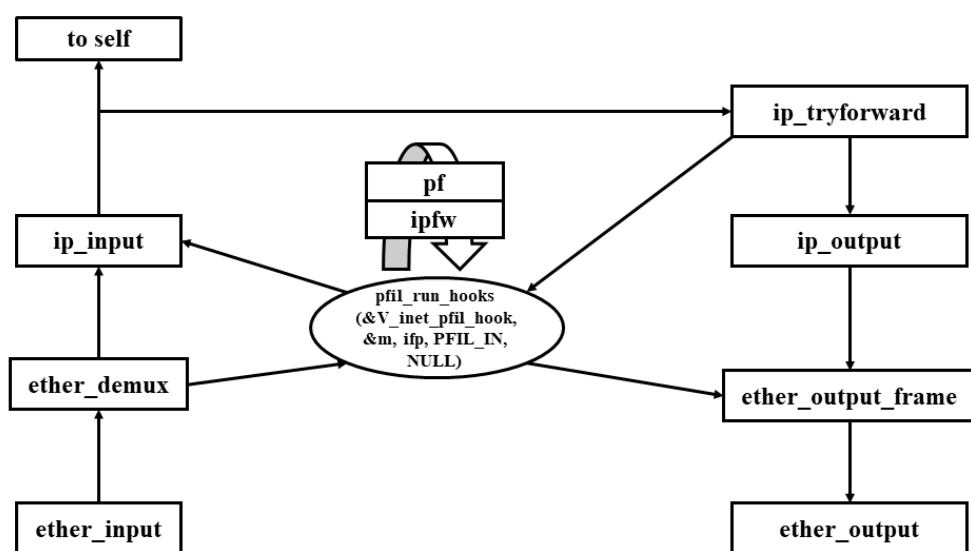


图 2-7 ipfw 数据包处理流程

`ipfw` 在数据链路层和网络层均注册了挂载点,在数据链路层调用 `ipfw_check_frame` 函数,尝试匹配数据链路层规则,对数据包作相应接受、丢弃、二层转发等操作;在网络层调用 `ipfw_check_packet` 函数,尝试匹配三层规则,对数据包作相应接受、丢弃、三层转发等操作,若是私有网段数据包则需要经过 NAT 处理,改写源/目的 `ip` 地址。

## 2.4 本章小结

本章主要介绍了基于 DPDK 的高性能防火墙所运用的主要技术。首先对 DPDK 的总体架构和提升性能的三个关键技术——轮询技术、CPU 亲和技术进行介绍,然

# 华中科技大学硕士学位论文

---

后简要描述了基于 DPDK 实现的高性能网络框架 f-stack 的工作机制，最后对两种典型的防火墙架构——Netfilter/iptables 架构和 Pfil/ipfw 架构进行详细分析，得出数据包处理低效的原因，为下一步的工作奠定理论基础。

## 3 基于 DPDK 的防火墙方案设计

本章主要介绍基于 DPDK 的高性能防火墙的方案设计。首先从功能性和非功能性两方面分析用户需求，确定开发方向；然后基于需求分析的结果本课题进行总体设计，并划分主要功能模块；最后根据总体设计中的模块划分对各个模块进行详细设计。

### 3.1 需求分析

#### 3.1.1 功能性需求

基于 DPDK 的高性能防火墙系统旨在为企业提供高速网络环境下安全策略的实施，实现基于五元组的网络数据包安全过滤，提供网络的访问控制、内部网络的监控与审计，禁止不全的网络服务，防止非法及未授权的网络通信进出被保护的企业内网，保证企业内部网络环境的安全，起到安全网关的作用。

针对以上企业对防火墙的业务需求，可以明确本系统的功能需求。本系统主要对数据包进行解析，根据访问控制规则对五元组信息进行匹配，对于符合规则的数据包，执行相应的转发、丢弃、通过等动作。同时，为了降低系统的丢包率，提高系统数据包转发速度，本系统增加了一条快转路径，对于允许转发的数据流提供快速进出防火墙系统的通道。为了保证防火墙功能的完整性、提供基本的 NAT、路由等功能，本文提供基于 `frebsd` 的协议栈路径，满足企业用户的多样化需求。在系统的维护与故障排查方面，提供日志模块，方便系统崩溃后的排查与快速恢复；为了便于企业对系统管理，本系统还设计了可视化前台页面，提供基于 Web 页面的系统交互，方便对访问规则的设置与修改，实现对企业网络的管控。

#### 3.1.2 非功能性需求

对于本防火墙系统的需求除了上述的功能性需求外，更加重要的是系统的非功

性能需求，旨在提升传统防火墙的性能，包括防火墙的转发速率、并发连接数等。

本课题着力于实现高性能防火墙，因此提升防火墙性能是至关重要的非功能性需求，防火墙的性能评估主要包括转发速率、并发连接数、新建连接速率等。为了方便企业用户对防火墙进行规则设置，本系统还需要提供与用户交互的 Web 页面。为了使系统在黑盒测试中不容易崩溃，还需要确保系统的鲁棒性维持系统稳定。

## 3.2 总体设计

基于 DPDK 的高性能防火墙的总体架构如图 3-1 所示。

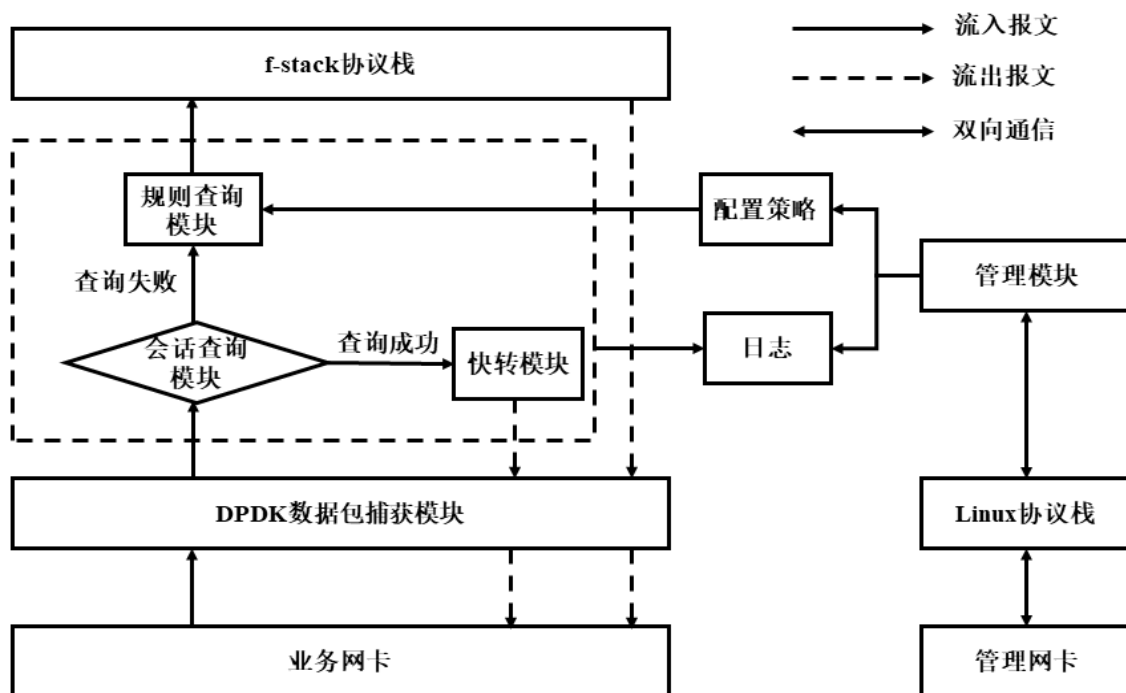


图 3-1 系统总体架构

本系统分为数据包捕获模块、会话查询模块、规则查询模块、快转模块、协议栈路径模块、管理模块。

其中数据包捕获模块、会话查询模块、规则查询模块、快转模块、协议栈模块的流量通过业务网卡，实现防火墙系统的业务通信，提升防火墙性能；管理模块通过 Linux 协议栈对系统进行配置和管理，用户可以通过管理网卡对系统进行远程操作。

## (1) 数据包捕获模块

本系统的数据包捕获模块是基于 DPDK 实现的，DPDK 提供了数据包处理函数库和驱动集合，为实现数据包捕获功能模块提供接口，便于将数据面和控制面的操作进行整合以提高数据包处理效率。

## (2) 会话查询模块

从 DPDK 捕获到的数据包中解析出源 IP 地址、目的 IP 地址、源端口、目的端口、协议类型信息后进行会话查询，若查询失败则尝试规则匹配，规则允许的情况下则新建会话；若查询成功则进入快转路径。在会话查询模块中，将 IP 报文的源 IP 地址、目的 IP 地址、源端口、目的端口、协议类型作为一个五元组，计算出哈希值用以标识数据流，利用哈希查找算法查询该流是否在会话表中。

## (3) 规则查询模块

对数据包进行规则查询时，采用基于分治法的规则匹配算法，将规则集按一定的分类方案分为小规则集，将数据包按规则的优先级的从高到低的顺序依次尝试匹配小规则集中的规则，若匹配成功则执行规则指定的动作且不再尝试匹配后面的规则，否则一直尝试匹配下一条规则直到最后优先级最低的默认规则匹配完毕，返回查询结果，若规则允许则将数据包交给上层协议栈处理。

## (4) 快转模块

快速转发使用 5 元组（源 IP 地址、源端口号、目的 IP 地址、目的端口号、协议号）来标识一条数据流。当一条数据流的第一个报文通过查找路由表转发后，将转发信息记录到会话表中，该数据流后续报文的转发就可以通过直接查找会话表进行转发。这样便大大缩减了 IP 报文的排队流程，减少报文的转发时间，提高 IP 报文的转发速率。

## (5) 协议栈模块

协议栈模块需要处理的数据包包括接收的第一个数据包、MPLS 包、ARP 包、VLAN 包等。协议栈模块还需要记录每个新建会话的信息包括目的 MAC 地址和数

据包的转发端口，便于快速转发数据包。除此之外用户态协议栈还提供路由查找功能。

## (6) 管理模块

数据包在经过会话查询模块、快转模块、规则查询模块时将包头信息和数据包流向都将被记录到日志文件，Web UI 管理页面加载了日志模块，而且方便用户进行规则配置。

## 3.3 详细设计

### 3.3.1 数据包捕获模块

本系统旨在高速网络环境下提高防火墙处理数据包的性能，数据包捕获模块主要负责将接收到的数据包从内核态映射到用户态，数据包捕获模块的性能直接影响整个系统的性能。DPDK 提供了数据包处理函数库和驱动集合，便于将数据面和控制面的操作进行整合以提高数据包处理效率。

由于 DPDK 是多核多线程架构，可以并行地捕获数据包，本系统的数据包捕获模块设计如图 3-3 所示。

本系统利用 CPU 亲和性将各个执行线程与不同的 CPU 逻辑核进行绑定，单独为每个线程分配一个 CPU 逻辑核，进而将网卡接收的报文分配到各个 CPU 上处理，减少单个 CPU 核心的处理压力，同时，由于 CPU 上运行的单个线程，并且每个逻辑 CPU 用于存储报文的内存池相互独立，无需因多个线程访问同一个报文队列而对队列进行加锁，减少了锁开销，可以减少 CPU 中断和资源竞争。

具备多队列的网卡，可以利用 RSS 技术对数据流计算 hash 值，不同的数据流按照一定的分发策略选择不同的队列，进而交给特定的 CPU 线程处理，可以实现使接收报文在多个 CPU 之间负载均衡、按流分发。同时，由于 DPDK 的网卡驱动借助工作在用户态，利用轮询模式批量接收网络数据包，通过 UIO 技术映射到用户态供应用程序使用，无需像传统协议栈的工作方式，对数据包进行内存拷贝，大大提高了数

据包捕获性能。

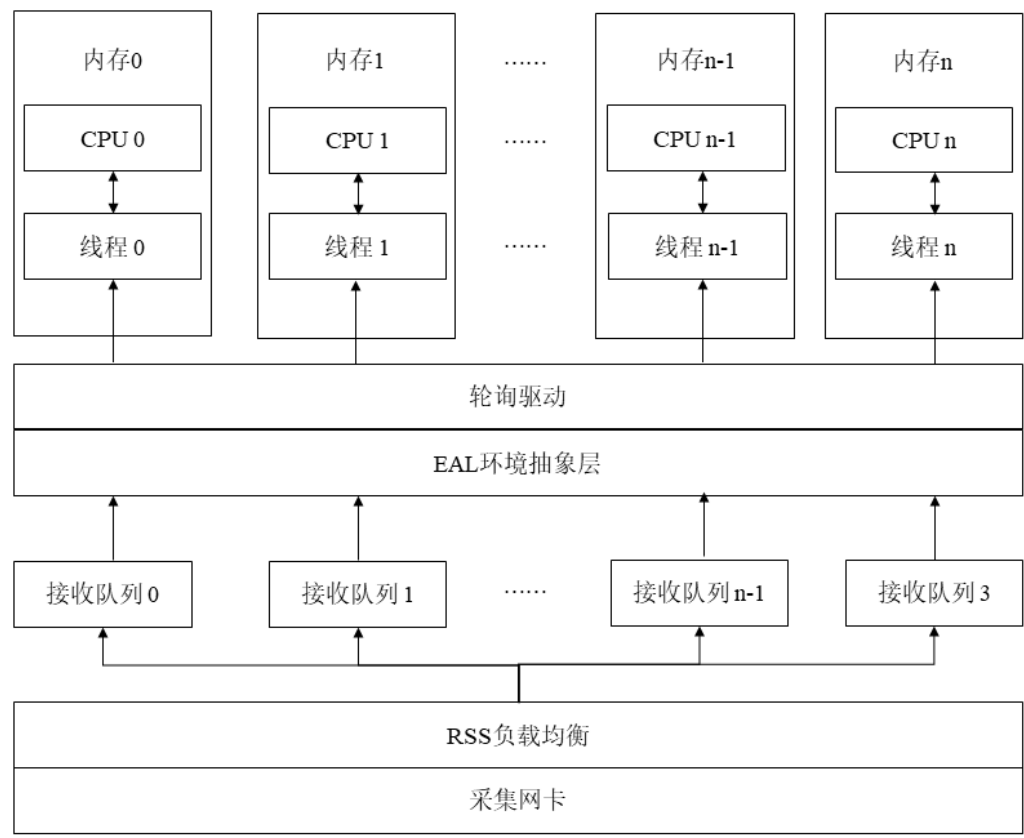


图 3-2 数据包捕获模块设计

3.3.2 会话查询模块

会话查询模块包括会话的数据结构设计，会话表的维护和会话查询算法三个部分。

(1) 会话的数据结构设计

当数据包尝试与某条会话进行匹配时，只有当数据包属于这条会话对应的数据流时才能匹配成功，则说明会话的数据结构中必须具有能唯一识别数据流的成员，而数据包包头中的源 IP 地址、目的 IP 地址、协议类型、源端口、目的端口这 5 个字段构成的五元组可以唯一标识数据流，因此会话结构中需要有五元组。

由系统的总体设计可知数据包经过会话查询后如果匹配成功需要将数据包转到

快转模块，快转模块也需要通过某种方法判断数据包的处理方式，数据包可能被快速转发出去，也可能要被交到协议栈处理。解决这个问题的方式是在会话结构中设置快转标志位，为 1 时直接转发数据包，为 0 时将数据包交给协议栈作后续处理。

一条数据流在经过防火墙系统时，为了不必每次都查询报文过滤规则和路由规则，需要在流表信息中添加转发出去的网卡端口和目的 MAC 地址，以便后续该流的数据包进行快速转发。

考虑到会话表的维护，应该需要通过一种标识来对会话表进行维护，当这个标识满足一定条件，就删除该条会话，保证会话表不会浪费存储空间。其中一种方案是设置会话时间戳，即会话的最后通信时间，每次快转模块或协议栈处理了属于该会话的数据包，就更新会话的时间戳为当前时间。

综上分析可知，会话数据结构中需要的成员包括五元组信息、快转标志位、数据包流出网卡端口，目的 MAC 地址，以及时间戳。

## (2) 会话表的维护

系统在运行过程中，随着时间推移，处理的会话也越来越多，如果不及时删除长时间没有数据包流入的会话，就会造成会话表中有大量空闲会话连接，不仅浪费空间资源，还会影响查询效率。因此要设定一个时间段，定期检查会话表，并且要规定会话的超时时间，一旦超时则从会话表中删除该条会话。通过对会话表的维护来保证系统的查询会话表的性能维持在较高水平。

## (3) 会话查询算法

本课题选用哈希算法作为会话查询算法，本小节将介绍两种典型的解决冲突的哈希算法并结合应用场景分析选择相对更佳的算法。

### 1) 链地址法

基于链地址法的哈希算法（拉链法）就是将链表和数组相结合，数组的每个元素是一个链表，若遇到哈希冲突，将冲突值加到相应的链表中。其数据结构如图 3-3 所示。



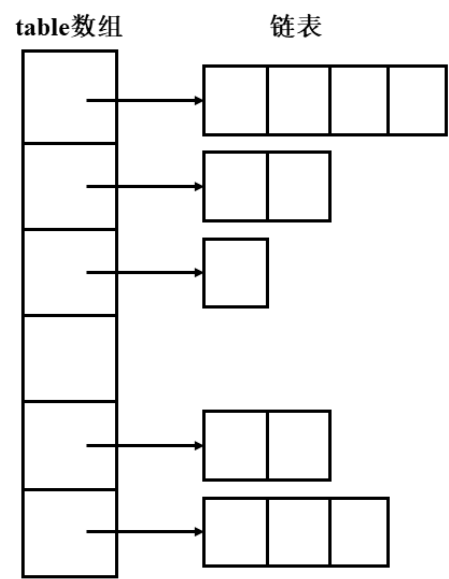


图 3-3 拉链法

2) 开放定址法

开放定址法与链地址法的不同之处在于需要存储的元素直接存放到数组中而不是数组的链表中。当遇到哈希冲突时，再按线性探查、二次探查、伪随机探测等方式形成探测序列，查找探测序列中下一个空数组单元。

3) 算法对比

本课题应用在高速网络环境中，部署在企业内部，考虑到需要支持企业内部大量会话连接，会话表规模较大。基于此应用场景总结对比了两种方式如表 3-1 所示。

表 3-1 算法对比

|       | 冲突处理                   | 平均查找长度 | 空间使用                       | 结点增删操作             |
|-------|------------------------|--------|----------------------------|--------------------|
| 开放定址法 | 容易产生堆积问题，不适于大规模的数据存储   | 较长     | 结点规模较大时浪费很多空间              | 遇冲突时实现较复杂，需处理后面的元素 |
| 链地址法  | 处理方式简单，无堆积现象，适合大规模数据存储 | 较短     | 结点规模较大时，增加的指针域可忽略不计，因此节省空间 | 实现方便，只需调整指针域       |

综上所述，结合本课题的应用场景将选用链地址法实现会话查询算法来保证会话查询性能。

3.3.3 规则查询模块

防火墙规则集中的规则包括源/目的 IP 地址、源/目的端口、源/目的 MAC 地址、协议类型以及对数据包的处理动作。传统防火墙规则匹配算法是顺序匹配，对收到的数据包与规则集中的规则尝试逐一匹配。随着规则集规模变大，导致规则查询的开销变大，防火墙性能降低。在这种情况下可以从两个方面着手改善规则查询的性能：①减少数据包需要匹配的数据集个数；②缩短每个数据包在每个规则集中的匹配时间。

本文运用基于分治思想的规则匹配算法，从以上两个方面提高规则查询效率。将规则集根据协议类型分成多个子规则集，这样将减少数据包匹配的规则集个数；再将子规则集中的规则按关联性分成有序和无序两组，根据两组规则各自的匹配特点使用各自适合的查询算法，如此优化也缩短了每个数据包在每个规则集中的匹配时间。

其中，规则分组算法涉及的规则的关联性用一个规则集的例子解释，规则集如表 3-2 所示。

防火墙过滤规则的主要包括源 IP 地址、目的 IP 地址、源端口、目的端口、协议类型、动作这 6 个域。其中源 IP 地址、目的 IP 地址、源端口、目的端口、协议类型这 5 个域用来作为数据包匹配的条件，动作域用来对匹配成功的数据包进行处理，包括拒绝和允许数据包通过。

表 3-2 规则集示例

| 规则 | 源 IP 地址      | 目的 IP 地址     | 源端口 | 目的端口 | 协议类型 | 动作     |
|----|--------------|--------------|-----|------|------|--------|
| R1 | 192.168.1.20 | 0.0.0.0      | 20  | 80   | TCP  | deny   |
| R2 | 192.168.1.0  | 0.0.0.0      | 20  | 80   | TCP  | accept |
| R3 | 172.18.28.50 | 172.18.28.60 | 22  | 21   | UDP  | accept |
| R4 | 192.168.1.20 | 0.0.0.0      | 20  | 80   | TCP  | accept |

# 华中科技大学硕士学位论文

|    |             |             |         |    |     |        |
|----|-------------|-------------|---------|----|-----|--------|
| R5 | 192.168.1.0 | 0.0.0.0     | 30      | 80 | TCP | accept |
| R6 | 192.168.1.0 | 172.18.28.0 | 0~65535 | 80 | TCP | accept |

在规则 R1 和 R2 中, R1 的源 IP 地址域是 R2 源 IP 地址域的子集, 其他条件域相等, 动作域互斥。当源 IP 地址为 192.168.1.20 的数据包到来时, 先匹配 R1, 数据包被拒绝, 不会再匹配 R2。若 R1 和 R2 的顺序交换, 同样的数据包就会先匹配到 R1 后被放行且不再匹配 R2。这种情况下认为 R1 的域是 R2 的子集, 定义两者是包含相关的关系。

在规则 R1 和 R3 中, 没有任何一个域具有包含关系或相等关系, 两者顺序互换也不会影响数据包的处理结果。这种情况下定义两者是完全无关的关系。

在规则 R1 和 R4 中, 两者所有条件域都相同, 认为两者完全相等。这种情况在实际的防火墙的规则部署中是不允许的。

在规则 R1 和 R5 中, R1 的源 IP 地址域是 R5 的子集, 而源端口域互斥。当源 IP 地址为 192.168.1.20, 源端口为 20 的数据包到来时, 先匹配 R1 被拒绝后, 不会再匹配 R5。两者顺序互换, 同样的数据包由于源端口不同则不会匹配到 R5 而还是匹配到 R1, 数据包被拒绝, 即数据包的处理结果与两者的匹配顺序无关。这种情况下定义两者是部分无关的关系。

在规则 R1 和 R6 中, R1 的源 IP 地址域和源端口域是 R6 的子集, 目的 IP 地址域是 R6 的超集, 其他条件域相同。当源 IP 地址为 192.168.1.20, 目的地址是 172.18.28.12, 源端口是 20 的数据包到来时, 先匹配到 R1, 数据包被拒绝, 不再匹配 R6。两条规则互换顺序后, 同样的数据包先匹配到 R6, 数据包被放行, 不再匹配 R1。这种情况下定义两者是交叉相关的关系

根据以上的分析可以给规则间的关系进行形式化的定义。

定义 1: 对于规则  $R_A$  和  $R_B$ , 如果  $R_A$  的任意条件域都是  $R_B$  对应域的子集, 则  $R_A$  和  $R_B$  包含相关。形式化表达为:

$\forall i \in \{sip, dip, sport, dport, protocol\}$  , 均有  $R_A[i] \subseteq R_B[i]$  , 且  $\exists i \in \{sip, dip, sport, dport, protocol\}$  使得  $R_A[i] \neq R_B[i]$  。

定义 2: 对于规则  $R_A$  和  $R_B$  , 如果  $R_A$  的任意条件域都不是  $R_B$  对应域的子集和超集, 则  $R_A$  和  $R_B$  完全无关。形式化表达为:

$\forall i \in \{sip, dip, sport, dport, protocol\}$  , 均有  $R_A[i] \subseteq R_B[i]$  和  $R_A[i] \supseteq R_B[i]$  均不成立。

定义 3: 对于规则  $R_A$  和  $R_B$  , 如果  $R_A$  的任意条件域与  $R_B$  对应域相等, 则  $R_A$  和  $R_B$  完全相等。形式化表达为:

$\forall i \in \{sip, dip, sport, dport, protocol\}$  , 均有  $R_A[i] = R_B[i]$  成立。

定义 4: 对于规则  $R_A$  和  $R_B$  , 如果  $R_A$  至少存在一个条件域是  $R_B$  对应域的子集或超集, 且至少存在一个域不是  $R_B$  对应域的子集或超集, 则  $R_A$  和  $R_B$  部分无关。形式化表达为:

$\exists i \in \{sip, dip, sport, dport, protocol\}$  , 使得  $R_A[i] \subseteq R_B[i]$  或  $R_A[i] \supseteq R_B[i]$  成立, 且  $\exists j \in \{sip, dip, sport, dport, protocol\}$  , 使得  $R_A[j] \subseteq R_B[j]$  和  $R_A[j] \supseteq R_B[j]$  均不成立。

定义 5: 若规则  $R_A$  和  $R_B$  , 如果  $R_A$  至少存在一个条件域是  $R_B$  对应域的子集, 且  $R_A$  的其他条件域是  $R_B$  对应域的超集, 则  $R_A$  和  $R_B$  交叉相关。形式化表达为:

记集合  $S = \{sip, dip, sport, dport, protocol\}$  ,  $\exists T \subset S$  ,  $\forall i \in T$  , 都有  $R_A[i] \subseteq R_B[i]$  , 且  $\forall j \in S - T$  , 都有  $R_A[j] \supseteq R_B[j]$  , 且  $\exists k \in S$  使得  $R_A[k] \neq R_B[k]$  。

根据上文的分析, 可知防火墙的过滤规则之间的关系有包含相关、交叉相关、部分无关和完全无关 (完全相等的规则在实际防火墙配置中不存在)。其中包含相关和

交叉相关的规则在改变匹配顺序后数据包的处理结果可能不同，因此为了保证包过滤的准确性，包含相关和交叉相关的规则要按原始规则集中的顺序执行；而部分无关和完全无关的规则对数据包的处理结果与匹配顺序无关。因此将规则集中的规则分为有序组和无序组。

### 3.3.4 快转模块

快转模块处理的是已经建立会话且该条数据流无需通过协议栈处理的数据包。为了实现这类数据包的快速转发，就需要减少获取数据包转发信息所需的时间，决定数据包转发的信息包括数据包流出网卡的端口和下一跳路由的 MAC 地址（即数据包的目的 MAC 地址），这两个信息都被记录在会话信息中，因此只需对数据包进行一次会话查询就可以获取到转发信息，然后根据信息将数据包直接转发出去。

快速转发的方式避免了很多不必要的协议栈处理流程例如查找路由、经过内核注册的钩子函数等，而且大大缩减了数据包的排队时延，减少转发时间，提高了防火墙的转发速率。

### 3.3.5 管理模块

数据包在经过会话查询模块、快转模块、规则查询模块时将包头信息和数据包流向都将被记录到日志文件，方便对系统的管理和维护。管理模块提供人机交互界面，使用 php 语言开发服务程序，处理用户提交的规则表单，并将规则写入到防火墙系统的规则文件中，将程序部署到 nginx 服务器中，再以 Web 页面的形式呈现给用户，提供对防火墙规则的增、删、改、查等功能，便于用户对网络策略实施进行管理。

## 3.4 本章小结

本章首先从功能性和非功能性两方面分析基于 DPDK 的高性能防火墙系统的用户需求，根据用户需求设计系统总体框架，并给出总体设计图来展示模块划分方式，最后对划分的模块介绍详细设计，数据包捕获模块中介绍了 DPDK 的网卡队列、线

程、CPU 逻辑核、内存等资源分配的原则；会话查询模块分为会话数据结构设计、会话表的维护、会话查询算法三个部分；规则查询模块分析了提高查询效率的两个方面，并介绍了规则间的 5 种关系，确定使用基于分治法的分组规则查询算法；快转模块分析了快速转发通过获取会话信息实现，对转发性能有提升作用；管理模块通过设计一个 Web UI 交互界面便于用户对系统进行管理。

## 4 基于 DPDK 的防火墙实现

本章将介绍基于 DPDK 的高性能防火墙系统的实现，对系统主要功能模块的内部逻辑进行详细介绍，通过实现系统为第 5 章的系统测试打下基础。

### 4.1 数据包捕获模块实现

DPDK 提供了与数据包捕获相关的接口，主要的数据结构如图 4-1 所示。

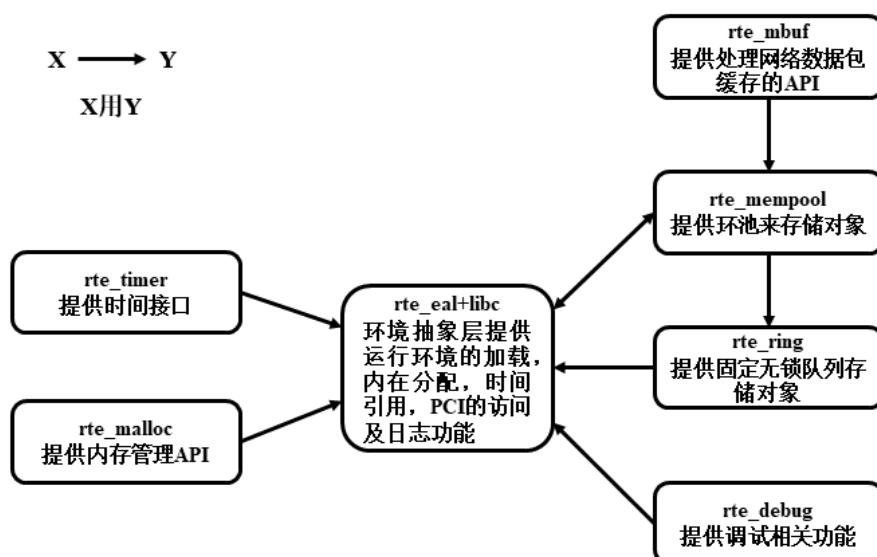


图 4-1 DPDK 包捕获模块集合

要实现数据包捕获模块，处理流程包括捕获机制初始化和数据包接收处理功能实现。数据包捕获模块流程图如图 4-2 所示。

捕获机制初始化中首先调用 `rte_eal_init()` 初始化环境抽象层 (Environment Abstraction Layer, EAL)；然后调用 `rte_eth_dev_init()`、`init_mbuf_pools()`、`init_port()` 分别初始化网卡、内存和端口；再根据一个网卡队列对应一个线程，一个线程绑定一个 CPU，每个 CPU 拥有独立的内存池的原则进行分配，调用 `rte_eth_rx_queue_setup()` 为每个网卡接收队列设置环形无锁队列 `rte_ring` 来存储对象，调用 `eal_thread_set_affinity()` 使得线程独占某 CPU 逻辑核，调用 `ixgbe_dev_rss_reta_query()`

设置 CPU 对应的接收队列；最后运行收包线程进入功能主体。

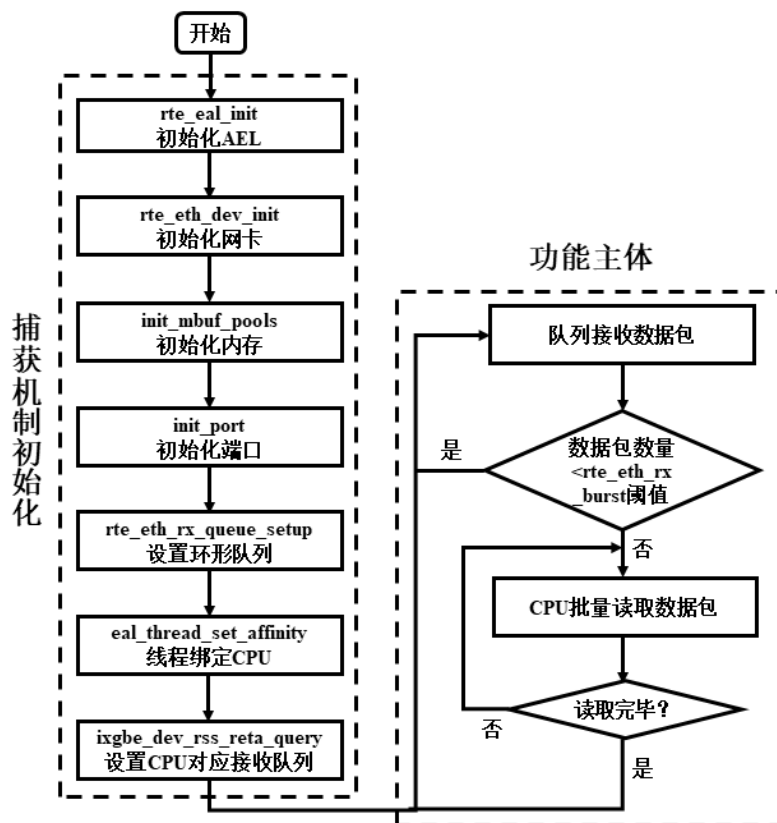


图 4-2 数据包捕获流程图

数据包接收处理功能运用了 DPDK 轮询技术提高数据包接收效率，同时为了 CPU 在读取数据包时减少读取内存的次数，运用 `rte_eth_rx_burst`，根据其设置的阈值 CPU 批量读取环形队列中的数据包。

## 4.2 会话查询模块实现

### 4.2.1 会话表的设计及维护

#### (1) 会话表设计

本课题的会话表设计为哈希表，以数组链表实现，存放经过系统的数据流，每一条数据流包含的信息如表 4-1 所示。



其中 sip、dip、proto、sport、dport 字段标识一个五元组；lasttime 字段代表的会话最后更新时间用于会话表维护；dportid 和 dmac 字段代表数据包从网卡转发出去的端口和目的 MAC 地址，对于同一会话中的数据包会根据这两个字段值将数据包快速转发出去；flag 字段标识是否对流进行快转，如果数据包属于已需要快转的数据流则将该数据包通过快转模块快速转发出去。

表 4-1 数据流信息

| 字段       | 意义         |
|----------|------------|
| sip      | 源 IP 地址    |
| dip      | 目的 IP 地址   |
| proto    | 协议类型       |
| sport    | 源端口        |
| dport    | 目的端口       |
| lasttime | 会话最后更新时间   |
| dportid  | 数据包流出端口    |
| dmac     | 目的 MAC 地址  |
| flag     | 会话建立标识     |
| next     | 指向下一条会话的指针 |

(2) 会话表维护

随着系统的不断运行，建立的会话连接不断增多，如果不及时维护会话表，会话表所占空间越来越大，很多长时间没有数据包流入的会话就会浪费内存资源，而且会减慢会话表的查询速度，对系统性能有很大影响，因此需要对会话表定期维护，保证系统性能稳定。

会话表的维护方式是每个整点维护一次，遍历会话表，若当前时间与会话最后更新时间之间的时间间隔过长，则删除该会话。且考虑到系统运行时间越长，会话表项

增多，处理时间会延长，导致获取的时间戳没有及时更新，存在误差，因此在查询一定数量的会话表之后需要更新时间戳。会话表的维护如图 4-3 所示。

首先获取当前时间  $cur$  并将已检查会话数  $count$  置为 0，若当前时间为整点，则从会话表的第一个会话开始检查，获取会话的  $lasttime$  字段，计算  $cur$  与  $lasttime$  时间差  $time$ ，如果  $time$  超过规定的时间  $t$  则说明该会话已经长时间未更新，因此删除该会话。无论会话是否删除，都要使已检查会话数  $count+1$ ，若已检查会话数达到阈值则重新获取当前时间并置  $count$  为 0，按此流程依次检查后面的会话。

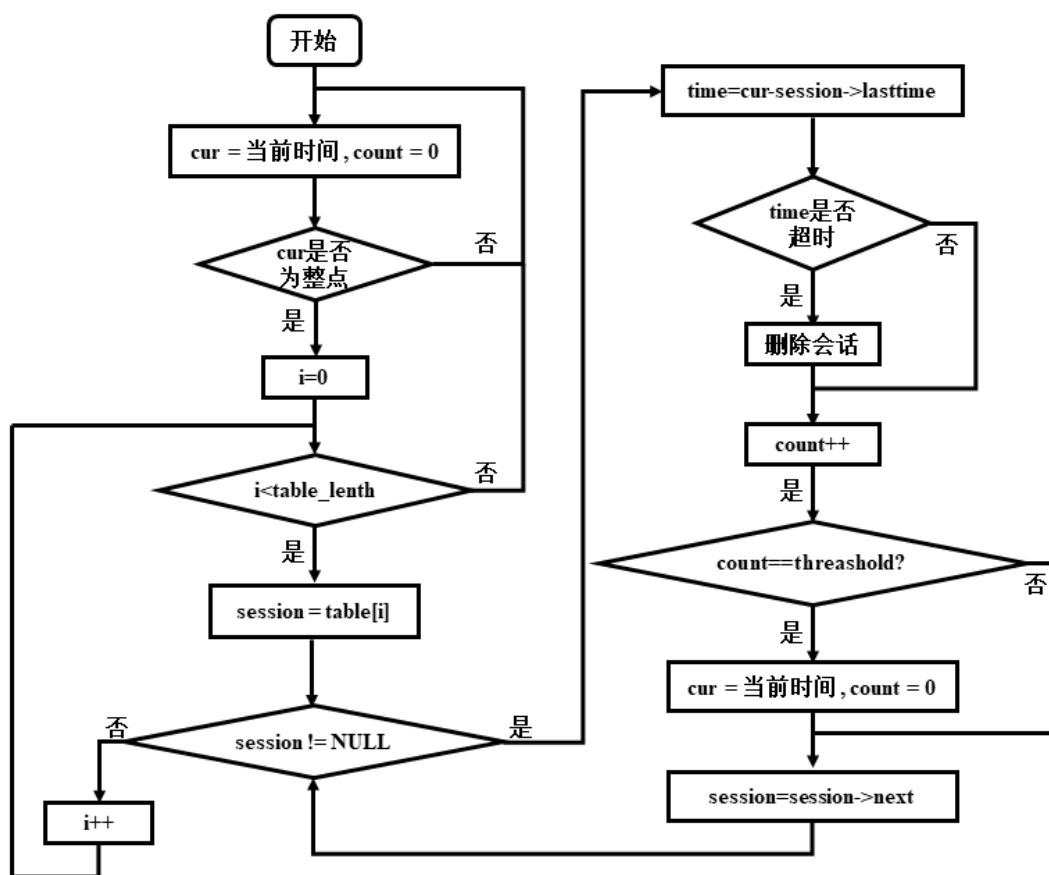


图 4-3 会话表维护

## 4.2.2 会话查询

根据第 3 章对会话查询算法的分析，本课题使用基于链地址法的哈希查询算法

进行会话查询，查询流程如图 4-4 所示。

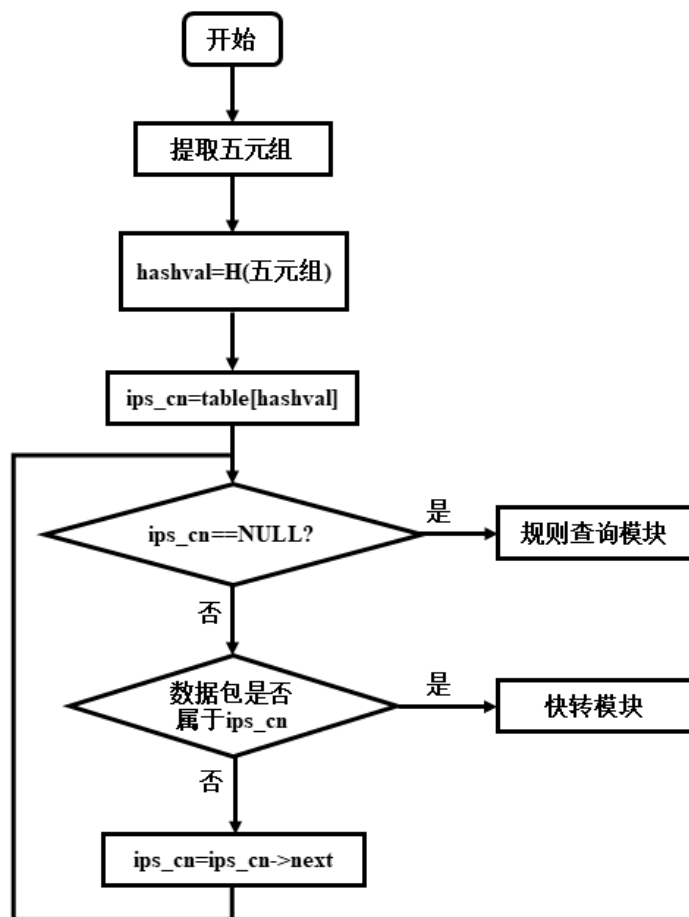


图 4-4 会话查询流程

首先对接收的数据包提取五元组，再对五元组用哈希函数处理后得到哈希值 hashval，查找会话表 ips\_cn\_table[hashval]上的会话链表，从链表的第一个会话开始查询，如果数据包五元组各字段与会话的五元组字段相同说明该数据包属于该会话，为了缩短数据包排队时延，将该数据包快速转发出去；如果遍历完链表后没有与该数据包匹配的会话，则将数据包交给规则匹配模块处理。

## 4.3 规则查询模块实现

本课题的规则表以链表方式实现，具体规则的字段及含义如表 4-2 所示。

表 4-2 规则字段及含义

| 字段             | 含义          |
|----------------|-------------|
| owner          | 用户          |
| priority       | 优先级         |
| direction      | 单向通信/双向通信   |
| mac            | 是否进行 MAC 匹配 |
| smac           | 源 MAC 地址    |
| dmac           | 目的 MAC 地址   |
| saddr          | 源 IP 地址     |
| smask          | 源 IP 掩码     |
| daddr          | 目的 IP 地址    |
| dmask          | 目的 IP 掩码    |
| src_port_start | 源端口号起始值     |
| src_port_end   | 源端口号结束值     |
| dst_port_start | 目的端口号起始值    |
| dst_port_end   | 目的端口号结束值    |
| proto          | 协议类型        |
| action         | 对数据包采取的动作   |
| valid          | 是否生效        |
| ttl            | 存活时间        |
| reference      | 引用次数        |
| next           | 指向下一条规则的指针  |

根据第 3 章对优化规则查询算法的分析，本课题选择基于分治法的规则查询算法，有两点原则：

## (1) 减少数据包匹配的规则集个数。

可以通过将规则集根据协议类型分成多个子规则集实现。根据协议类型将原始规则集中的所有规则分为 TCP 规则、UDP 规则、ICMP 规则、其他及无协议规则 4 类，匹配相应规则集处理数据包，流程如图 4-5 所示。

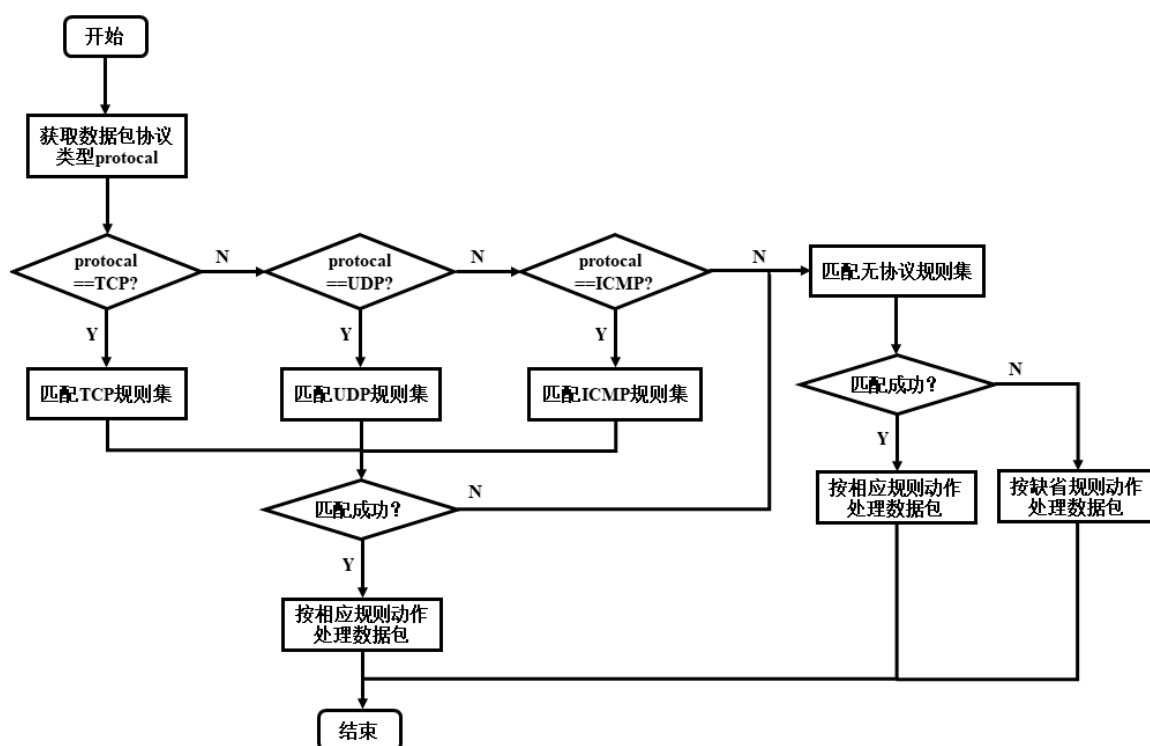


图 4-5 基于分治法的规则匹配流程

首先获取接收的数据包头部的协议类型信息，若协议字段的值为 6，则该数据包是 TCP 包，将其尝试与 TCP 规则集进行规则匹配；若协议字段为 17，则将该数据包尝试与 UDP 规则集进行规则匹配；若协议字段为 1，则该数据包尝试与 ICMP 规则集进行规则匹配；对于其他协议字段均将数据包与无协议规则集尝试进行规则匹配，并且在数据包在 TCP 规则集或 UDP 规则集或 ICMP 规则集中匹配失败时，再次将数据包尝试在无协议规则集中尝试进行规则匹配，根据最终匹配的结果对数据包进行相应的接收、丢弃或转发等处理。

## (2) 将子规则集中的规则按关联性分成有序和无序两组，根据两组规则各自的

匹配特点使用各自适合的查询算法。

根据 3.3.3 节对规则间关系的分析, 为了对规则分组, 重点是要清楚每个条件域的关系, 是真子集或相等或真超集或互斥, 因此需要一个数组 `cmp[5]` 存放每个条件域的关系。又因为只要有一个条件域互斥, 则这两条规则就是完全无关或部分无关, 因此设定 `cmp[i]` 的取值范围是 0 和 1, 0 代表互斥, 1 代表相关, 将数组 5 个元素相与, 如果结果为 0, 则添加到无序组中, 否则加到有序组中, 再对当前无序组和有序组中的规则不断分析调整, 最终得到的无序组中的所有规则互相都是完全无关或部分无关, 有序组中的规则交叉相关或包含相关。规则分组匹配算法流程如图 4-6 所示。

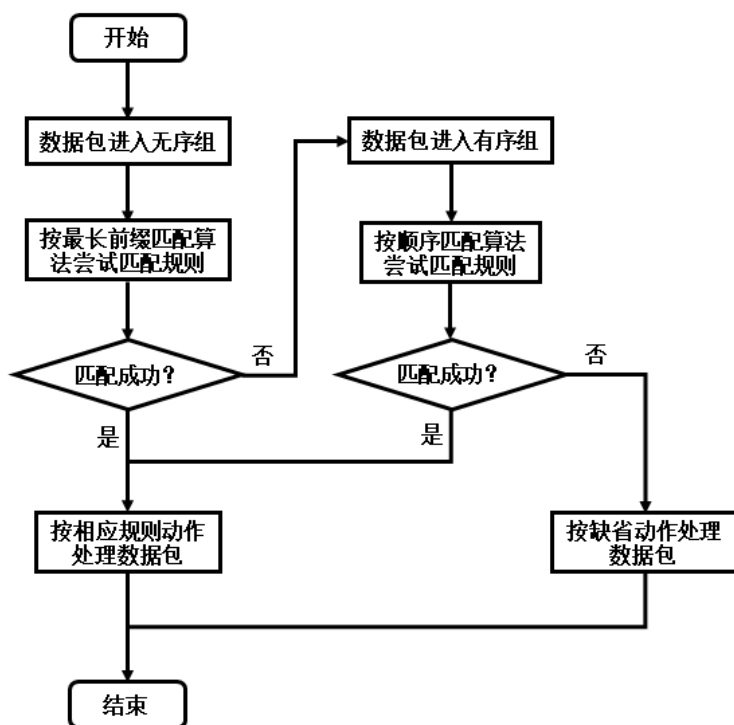


图 4-6 分组匹配算法

无序组的规则可以乱序匹配, 无序组的匹配方式如图 4-7 所示, 对无序组规则集中的规则, 首先根据源/目的 IP 地址以及掩码得到网络地址前缀, 采用 Radix Trie (基数) 树对源/目的 IP 地址段分别建立两棵 Radix Trie 树存放前缀信息, 即 IP 地址范围。查找时, 根据数据包的源/目的 IP 地址分别在这两棵 Trie 树中查找。将 IP 地址的比特串进行比较, 比特位为 0 时向左查找, 比特位为 1 时向右查找, 直到要比较

的比特位上没有分支，查找结束，得到的节点即为最长前缀节点。Radix 树是一棵二叉树，每比较一个比特位，地址空间减半，因此算法时间复杂度为  $O(\log n)$ 。对源/目的 IP 地址的最长前缀匹配可以同时进行，若得到多条规则，则对源 IP 地址最长前缀匹配结果与目的 IP 地址最长前缀匹配结果取交集，然后再对交集集中的规则进行顺序匹配。通过对源 IP 地址、目的 IP 地址这两个域进行前缀匹配再取交集的方式，可以大大缩小顺序匹配的范围，提高规则查询的速度。

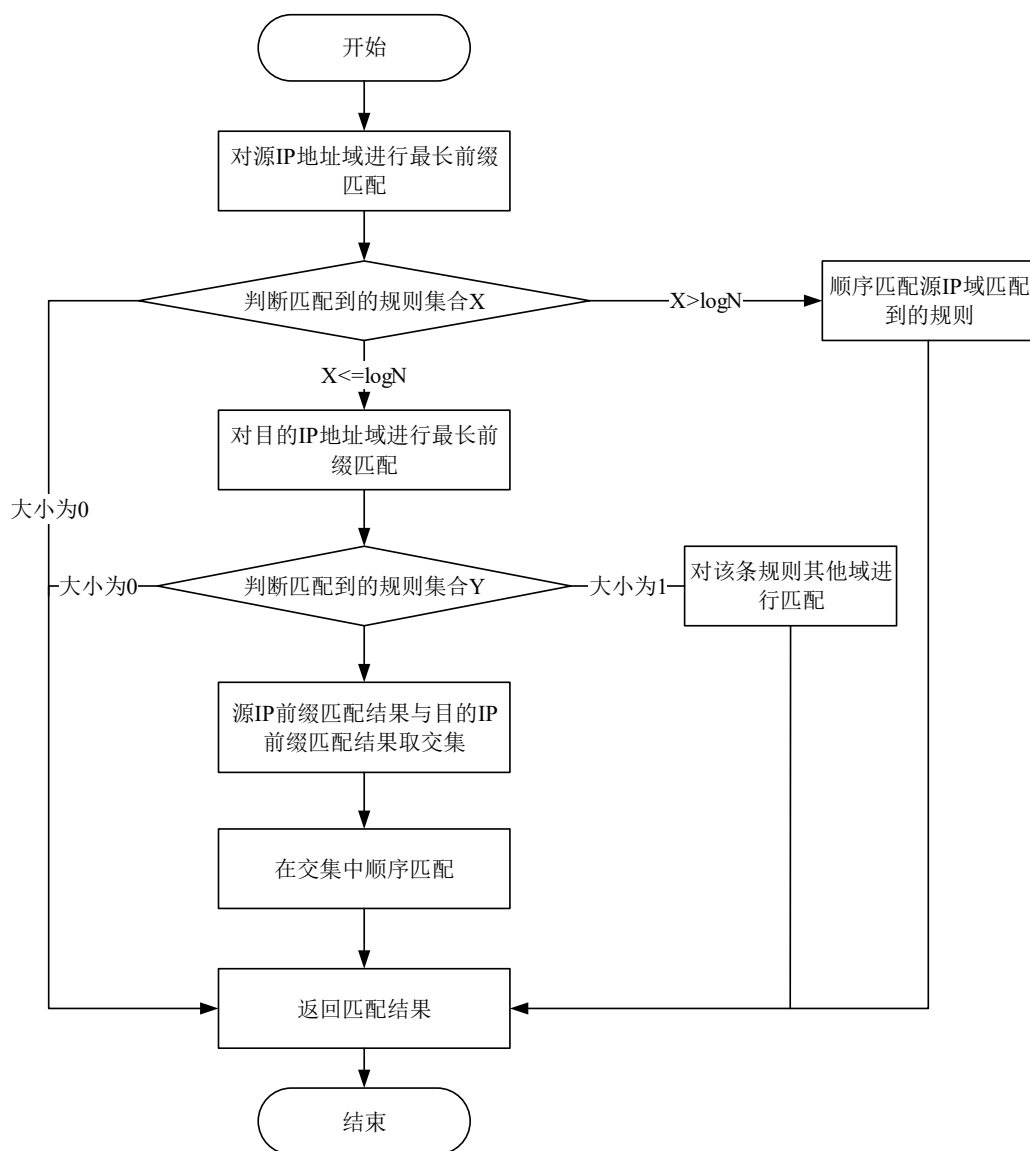


图 4-7 无序组匹配流程图

记无序组规则总数为  $N$ ，为了尽可能的减少查找次数，首先对源 IP 域进行前缀匹配，对匹配结果（记为集合  $X$ ）进行判断，若  $X$  大小为 0 则直接返回匹配结果（未匹配到规则），若  $X$  小于等于  $\log N$ ，则对集合  $X$  中的规则进行顺序匹配，若  $X$  大于等于  $\log N$ ，则再在目的 IP 域中进行前缀匹配，对匹配结果（记为集合  $Y$ ）进行判断，若  $Y$  大小为 0 则直接返回匹配结果（未匹配到规则），若  $Y$  大小为 1，则直接匹配集合中唯一一条规则的其他域。否则，求出集合  $X$  与集合  $Y$  的交集，然后在交集中，顺序进行规则匹配。

有序组的规则匹配顺序不可随意改变，所以用顺序查找对规则逐一尝试进行匹配。

实现规则匹配模块的查询功能后，还要建立模块与在系统内与其他模块的联系，如图 4-8 所示。

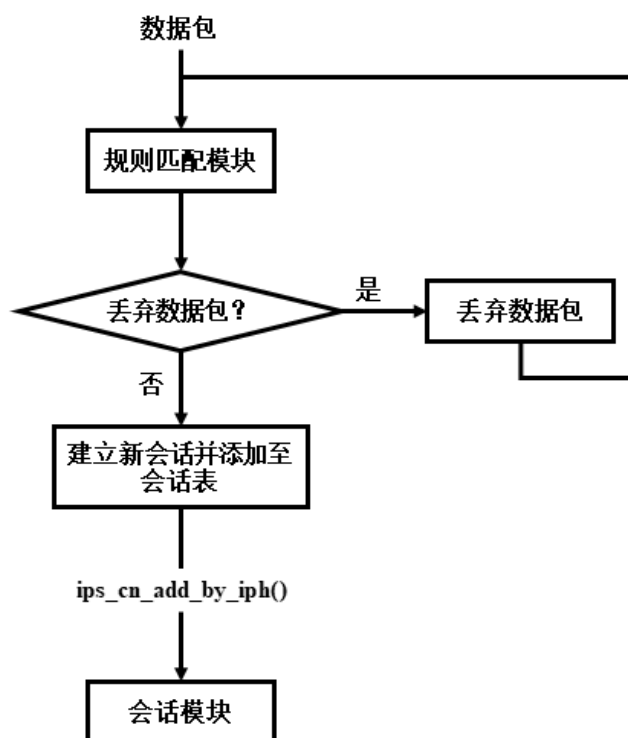


图 4-8 模块连接

数据包若经过规则匹配模块处理后，若规则设定接收或转发该数据包，则为该数



据包新建一个会话，并利用 `ips_cn_add_by_iph()` 会话模块接口将会话添加到会话表中。

## 4.4 快转模块实现

本系统设计快转模块的原因是：如果对于已建立会话的数据包还是要经过规则匹配，然后交给上层协议栈处理这一系列流程的话，可能会造成大量数据包排队等待处理的情况发生，这样会给系统带来额外的排队时延，造成系统的性能损耗。在这种情况下如果设置快转路径，根据已建立会话的相关信息如网卡转发端口（`dportid`）、目的 MAC 地址（`dmac`）等可以直接将数据包转发出去，有效减少了排队时延，保障系统性能稳定在较高水平。除转发数据包外，还能够更新会话的最后使用时间字段（`lasttime`），利于会话表的及时维护。快转模块流程如图 4-9 所示。

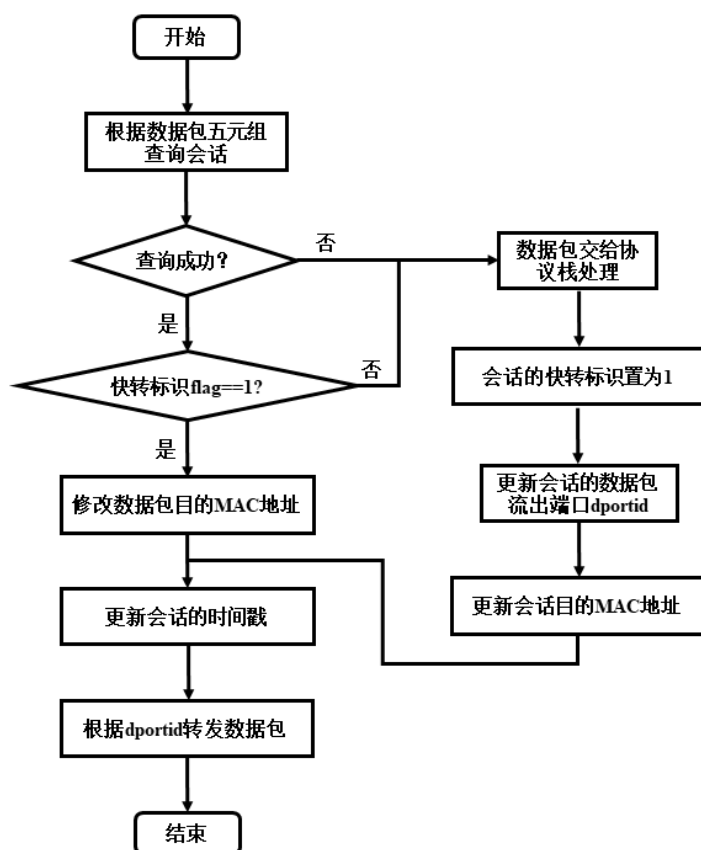


图 4-9 快转模块流程

首先根据数据包的五元组信息查询会话，如果没有匹配成功，说明该数据包是会

话的第一个包，成功新建会话后将该数据包交给协议栈处理，并更新会话信息，包括将快转标识 flag 置为 1，根据数据包的流出端口和目的 MAC 地址更新会话的 dportid 字段和 dmac 字段，更新会话的时间戳为当前时间。如果匹配成功，会话已经建立，则根据会话的 dmac 字段修改数据包的目的 MAC 地址，更新会话的时间戳并由会话 dportid 字段对应的网口将数据包转发出去，实现对已建立连接的数据包的快速转发功能。

## 4.5 管理模块实现

管理模块提供人机交互界面，使用 php 语言开发服务程序，将程序部署到 nginx 服务器中，再以 Web 页面的形式呈现给用户，操作简单，便于用户对系统的配置管理。本系统的管理模块提供以下功能：

(1) 登录认证：用户输入正确的用户名和密码完成登录认证后方可进入系统进行后续操作。如图 4-10 所示。



图 4-10 登录认证

(2) 规则配置管理：提供规则的增加、删除、编辑和查看功能。处理用户提交的

规则表单，并将规则写入到防火墙系统的规则文件中。如图 4-11 所示。

| 动态访问控制规则 |    |       |        |         |              |                 |               |                 |      |         |         |    | + 添加 |  |  | ▲ 编辑 |  | ■ 删除 |  |
|----------|----|-------|--------|---------|--------------|-----------------|---------------|-----------------|------|---------|---------|----|------|--|--|------|--|------|--|
| 序号       | 方向 | MAC匹配 | 源MAC地址 | 目的MAC地址 | 源IP          | 源掩码             | 目的IP          | 目的掩码            | 协议   | 源端口     | 目的端口    | 动作 | 有效时间 |  |  |      |  |      |  |
| 1        | 单向 | 否     | 无      | 无       | 192.168.1.13 | 255.255.255.255 | 192.168.2.2   | 255.255.255.255 | TCP  | 0-65535 | 0-65535 | 拒绝 | 永久有效 |  |  |      |  |      |  |
| 2        | 单向 | 否     | 无      | 无       | 192.168.1.0  | 255.255.255.0   | 192.168.3.0   | 255.255.255.0   | TCP  | 0-65535 | 0-65535 | 允许 | 永久有效 |  |  |      |  |      |  |
| 3        | 单向 | 否     | 无      | 无       | 192.168.1.6  | 255.255.255.255 | 172.18.28.110 | 255.255.255.255 | TCP  | 0-65535 | 80-80   | 拒绝 | 永久有效 |  |  |      |  |      |  |
| 4        | 单向 | 否     | 无      | 无       | 0.0.0.0      | 0.0.0.0         | 172.18.28.110 | 255.255.255.255 | ICMP | 0-0     | 0-0     | 拒绝 | 永久有效 |  |  |      |  |      |  |
| 5        | 单向 | 否     | 无      | 无       | 192.168.1.20 | 255.255.255.255 | 192.168.3.4   | 255.255.255.255 | UDP  | 0-65535 | 0-65535 | 拒绝 | 永久有效 |  |  |      |  |      |  |
| 6        | 双向 | 否     | 无      | 无       | 192.168.1.32 | 255.255.255.255 | 192.168.2.23  | 255.255.255.255 | UDP  | 0-65535 | 0-65535 | 拒绝 | 永久有效 |  |  |      |  |      |  |
| 7        | 双向 | 否     | 无      | 无       | 192.168.1.0  | 255.255.255.0   | 192.168.2.0   | 255.255.255.0   | IP   | 0-0     | 0-0     | 拒绝 | 永久有效 |  |  |      |  |      |  |
| 8        | 单向 | 否     | 无      | 无       | 192.168.2.5  | 255.255.255.255 | 172.18.1.3    | 255.255.255.255 | IP   | 0-0     | 0-0     | 拒绝 | 永久有效 |  |  |      |  |      |  |

图 4-11 规则配置

管理模块中的规则添加如图 4-12 所示，可以对五元组、mac 地址、选项进行添加或选择。

添加规则

☐ MAC匹配

源MAC:

目的MAC:

IP匹配

源IP:

源掩码:

目的IP:

目的掩码:

源端口:

0

65535

目的端口:

0

65535

选项

方向:

单向匹配

动作:

拒绝

协议:

TCP

有效时间:

0

秒(0为永久)

添加

关闭

图 4-12 规则添加

添加规则表单中的选项部分，方向可以选择单向匹配或双向匹配，动作可以选择拒绝/允许/忽略 ipsec，协议可以选择 TCP/UDP/ICMP/IP，有效时间可以设置任意时长，其中 0 秒表示永久，超时后该规则将失效。

(3) 会话查询：设置查询条件，从当前会话表中查询符合条件的会话项并显示结果。如图 4-13 所示。

o 查询条件

源地址: 0.0.0.0 / 0.0.0.0

目的地址: 0.0.0.0 /

☒ TCP: 源端口 0 - 65535 目的端口 0 - 65535

☒ UDP: 源端口 0 - 65535 目的端口 0 - 65535

☒ ICMP: ☒ 回送 ☒ 时间戳 ☒ 信息请求 ☒ 地址掩码请求

连接状态: ☐ 完全 ☐ 不完全

查询条数: 100 (1-500)

o 查询结果

| 协议  | 创建时间                    | 源IP           | 源端口/(ICMP)类型 | 目的IP          | 目的端口/(ICMP)类型 | 状态      | 最后通信时间                  |
|-----|-------------------------|---------------|--------------|---------------|---------------|---------|-------------------------|
| UDP | 2019-05-09 Thu 20:42:00 | 172.18.11.215 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:42:02 |
| UDP | 2019-05-09 Thu 20:42:00 | 172.18.11.216 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:42:01 |
| UDP | 2019-05-09 Thu 20:41:59 | 172.18.17.206 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:42:01 |
| UDP | 2019-05-09 Thu 20:41:58 | 172.18.11.212 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:42:01 |
| UDP | 2019-05-09 Thu 20:41:58 | 172.18.7.130  | 138          | 172.18.31.255 | 138           | SIMPLEX | 2019-05-09 Thu 20:41:58 |
| UDP | 2019-05-09 Thu 20:41:57 | 172.18.11.211 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:41:58 |
| UDP | 2019-05-09 Thu 20:41:55 | 172.18.11.209 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:41:58 |
| UDP | 2019-05-09 Thu 20:41:54 | 172.18.11.210 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:41:56 |
| UDP | 2019-05-09 Thu 20:41:53 | 172.18.11.205 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:41:59 |
| UDP | 2019-05-09 Thu 20:41:50 | 172.18.17.204 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:41:58 |
| UDP | 2019-05-09 Thu 20:41:42 | 172.18.9.240  | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:42:00 |
| UDP | 2019-05-09 Thu 20:41:40 | 172.18.17.205 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:42:00 |
| UDP | 2019-05-09 Thu 20:41:39 | 172.18.17.207 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:41:57 |
| UDP | 2019-05-09 Thu 20:41:33 | 172.18.11.203 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:42:02 |
| UDP | 2019-05-09 Thu 20:40:52 | 172.18.17.202 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:41:59 |
| UDP | 2019-05-09 Thu 20:40:48 | 172.18.17.201 | 137          | 172.18.31.255 | 137           | SIMPLEX | 2019-05-09 Thu 20:41:58 |

1 - 16 共 16 条

图 4-13 会话查询

(4) 统计会话：对系统建立的会话进行分类统计。如图 4-14、图 4-15 所示。

o 统计当前连接

刷新

| 项目         | 值      |
|------------|--------|
| 系统最大连接数    | 600000 |
| 缓冲连接数      | 25     |
| 当前活动连接数    | 19     |
| 完全TCP连接数   | 0      |
| 不完全TCP连接数  | 0      |
| 双向UDP连接数   | 0      |
| 单向UDP连接数   | 19     |
| 有应答ICMP连接数 | 0      |
| 无应答ICMP连接数 | 0      |

1 - 9 共 9 条

图 4-14 当前连接统计表

当前活动连接数

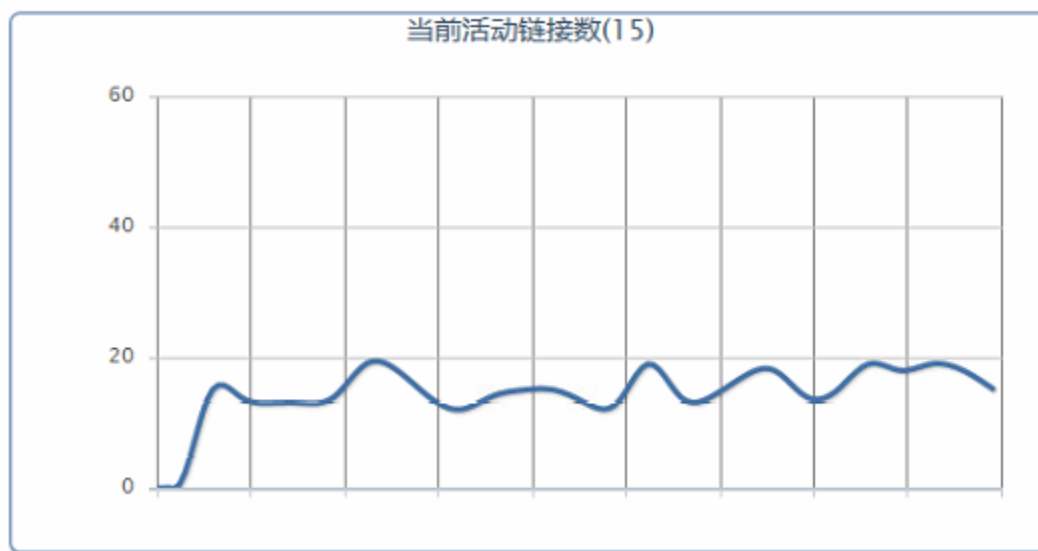


图 4-15 当前活动连接数

## 4.6 本章小结

本章介绍了基于 DPDK 的高性能防火墙系统各模块的详细实现。数据包捕获模块利用 DPDK 提供的数据包捕获接口实现,包括捕获机制初始化和数据包接收处理;会话查询模块的实现包括会话表的设计、用哈希算法实现会话查询算法、定时对会话表的维护;规则查询模块利用基于分治法的规则匹配算法实现;快转模块的功能是为了减少数据包排队时延,所以按照对已建立会话的数据包直接进行转发的逻辑来实现的;管理模块使用 php 语言开发服务程序,将程序部署到 nginx 服务器中,再以 Web 页面的形式呈现给用户。

5 基于 DPDK 的防火墙测试

本章介绍基于 DPDK 的防火墙系统的功能和性能测试。首先介绍功能测试，包括测试环境、测试方案、测试结果以及对测试结果的分析；然后介绍性能测试，在性能测试中先介绍测试环境，然后设计测试方案，接下来 4 小节具体介绍测试的内容和过程，包括 DPDK 二层转发性能、f-stack 三层转发性能、系统快转性能、系统整体性能；最后根据性能测试结果进行比较分析，得出结论。

5.1 功能测试

5.1.1 测试环境

本文利用 Virtualbox 虚拟机管理器在 Windows 10 系统上搭建测试环境，其中本防火墙系统安装在 Ubuntu 18.04 虚拟机中，该虚拟机安装有 3 块网卡，包括 1 块管理网卡和 2 块业务网卡，2 块业务网卡分别与内外网连接；内网即受保护的网路，内网中有一台安装了 Nginx 服务器，为 ubuntu14.04 虚拟机；外网由一台 PC 机模拟，也为 ubuntu14.04 虚拟机。所有虚拟机网卡采用 Host only 模式。本系统的功能测试的环境配置如表 5-1 所示。

表 5-1 功能测试环境配置

| 配置信息   | 规格                             |
|--------|--------------------------------|
| CPU    | AMD Ryzen 5 1600 CPU @ 3.20GHz |
| 内存     | 16GB DDR4                      |
| 操作系统   | Windows 10 64 位                |
| 虚拟机管理器 | Virtualbox 6.0                 |
| 客户端    | Ubuntu 14.04                   |
| 服务器    | Ubuntu 14.04                   |
| 防火墙    | Ubuntu 18.04                   |

## 5.1.2 测试方案

本测试中，防火墙系统所属虚拟机有 1 块管理网卡和 2 块业务网卡，业务网卡 IP 地址为 172.18.10.254 和 192.168.10.254，模拟外网的 PC 机( IP: 172.18.10.3 )的网关设为 172.18.10.254，内网服务器( IP: 192.168.10.254 )的网关设为 192.168.10.254，保证内外网之间的流量必须经过防火墙。功能测试的网络拓扑图如图 5-1 所示。

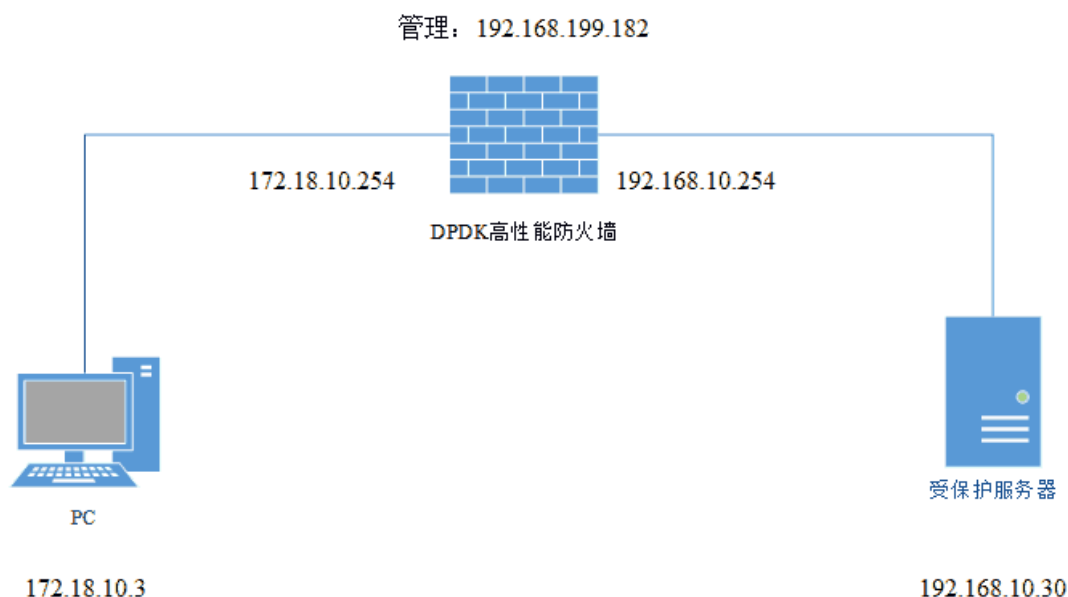


图 5-1 功能测试网络拓扑图

本次功能测试的内容包括包过滤功能测试和快转功能测试。包过滤测试分别对 TCP、UDP、ICMP 报文进行过滤功能测试；快转测试对已建立会话的数据包的流向进行跟踪。

(1) TCP 报文过滤：对从外网(172.18.10.3)发送到内网服务器(192.168.10.30)的 80 端口的 TCP 数据包进行过滤。

(2) UDP 报文过滤：针对从外网(172.18.10.3)发送到内网服务器(192.168.10.30)的数据包，过滤目的端口为 1024~2000 的 UDP 数据包。

(3) ICMP 报文过滤：过滤所有从外网(172.18.10.3)发送到内网网段 ( 192.168.10.0/24 )的 ICMP 数据包。

(4) 快转功能测试: 对外网(172.18.10.3)和内网(192.168.10.30)双向通信的数据包, 记录数据包的流向: 被协议栈接管处理或是被快速转发。

## 5.1.3 测试结果

根据上述测试方案对本防火墙系统的功能进行测试后得到测试结果。

### (1) TCP 报文过滤

配置过滤规则前能进行正常 TCP 通信, 客户端返回结果如图 5-2 所示, 日志记录结果如图 5-3 所示。

```
vagrant@ubuntu-client:~$ curl 192.168.10.30
<!-- Created by Zhongqi Xiao -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
</head>
<body>
  服务器的ip地址为: 192.168.10.30<br />浏览当前页面的客服端的ip地址为: 172.18.10.3<br />客户端真实ip地址: 172.18.10.3<br /></body>
</html>
vagrant@ubuntu-client:~$
```

图 5-2 过滤 TCP 前客户端结果

```
[2019-05-19 18:46:41]:[PROTOCOL=TCP]:172.18.10.3:48839 -> 192.168.10.30:80      match f-stack:to fstack freebsd...
[2019-05-19 18:46:41]:[PROTOCOL=TCP]:192.168.10.30:80 -> 172.18.10.3:48839      match f-stack:to fstack freebsd...
[2019-05-19 18:46:41]:find connection:
[2019-05-19 18:46:41]:[PROTOCOL=TCP]:172.18.10.3:48839 -> 192.168.10.30:80      match STOLEN:go to fast forwarding...
[2019-05-19 18:46:41]:find connection:
[2019-05-19 18:46:41]:[PROTOCOL=TCP]:172.18.10.3:48839 -> 192.168.10.30:80      match STOLEN:go to fast forwarding...
[2019-05-19 18:46:41]:find connection:
[2019-05-19 18:46:41]:[PROTOCOL=TCP]:192.168.10.30:80 -> 172.18.10.3:48839      match STOLEN:go to fast forwarding...
[2019-05-19 18:46:41]:find connection:
[2019-05-19 18:46:41]:[PROTOCOL=TCP]:172.18.10.3:48839 -> 192.168.10.30:80      match STOLEN:go to fast forwarding...
[2019-05-19 18:46:41]:find connection:
[2019-05-19 18:46:41]:[PROTOCOL=TCP]:172.18.10.3:48839 -> 192.168.10.30:80      match STOLEN:go to fast forwarding...
[2019-05-19 18:46:41]:find connection:
[2019-05-19 18:46:41]:[PROTOCOL=TCP]:192.168.10.30:80 -> 172.18.10.3:48839      match STOLEN:go to fast forwarding...
[2019-05-19 18:46:41]:find connection:
[2019-05-19 18:46:41]:[PROTOCOL=TCP]:172.18.10.3:48839 -> 192.168.10.30:80      match STOLEN:go to fast forwarding...
[2019-05-19 18:46:41]:find connection:
[2019-05-19 18:46:41]:[PROTOCOL=TCP]:192.168.10.30:80 -> 172.18.10.3:48839      match STOLEN:go to fast forwarding...
```

图 5-3 过滤 TCP 前日志记录结果

配置过滤规则后无法与目标端口通信, 客户端无法返回结果, 如图 5-4 所示, 日志记录结果如图 5-5 所示。



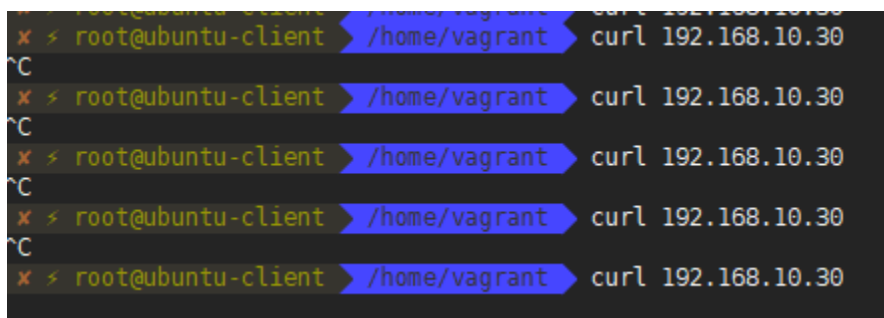


图 5-4 过滤 TCP 后客户端无法返回结果

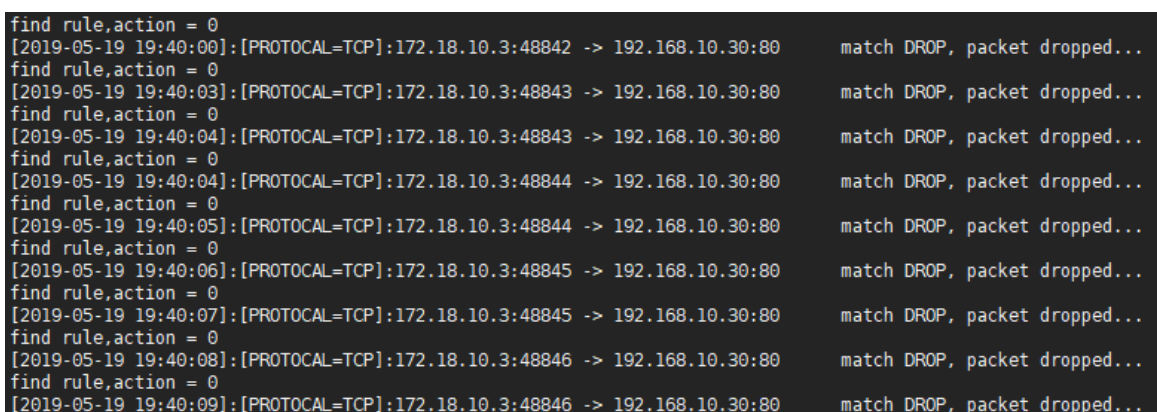


图 5-5 过滤 TCP 后日志记录结果

配置 TCP 过滤规则后，防火墙丢弃了从 172.18.10.3 的任意端口发往 192.168.10.30 的 80 端口的 TCP 数据包。

## (2) UDP 报文过滤

配置过滤规则前内外网可以进行正常的 UDP 通信，如图 5-6 所示，日志记录结果如图 5-7 所示。

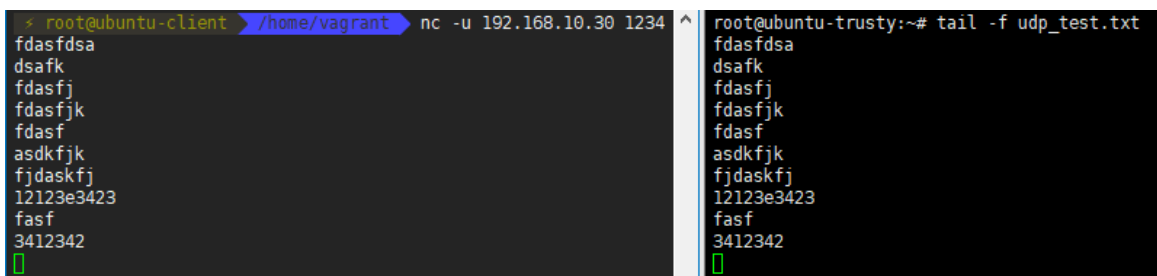


图 5-6 过滤 UDP 前内外网正常通信

```
[2019-05-19 19:49:42]:[PROTOCOL=UDP]:172.18.10.3:43355 -> 192.168.10.30:1234 match f-stack:to fstack freebsd...
match f-stack:to fstack freebsd...
match f-stack:to fstack freebsd...
match f-stack:to fstack freebsd...
match f-stack:to fstack freebsd...
[2019-05-19 19:49:43]:find connection:
[2019-05-19 19:49:43]:[PROTOCOL=UDP]:172.18.10.3:43355 -> 192.168.10.30:1234 match STOLEN:go to fast forwarding...
[2019-05-19 19:49:43]:find connection:
[2019-05-19 19:49:43]:[PROTOCOL=UDP]:172.18.10.3:43355 -> 192.168.10.30:1234 match STOLEN:go to fast forwarding...
[2019-05-19 19:49:44]:find connection:
[2019-05-19 19:49:44]:[PROTOCOL=UDP]:172.18.10.3:43355 -> 192.168.10.30:1234 match STOLEN:go to fast forwarding...
[2019-05-19 19:49:44]:find connection:
[2019-05-19 19:49:44]:[PROTOCOL=UDP]:172.18.10.3:43355 -> 192.168.10.30:1234 match STOLEN:go to fast forwarding...
[2019-05-19 19:49:45]:find connection:
[2019-05-19 19:49:45]:[PROTOCOL=UDP]:172.18.10.3:43355 -> 192.168.10.30:1234 match STOLEN:go to fast forwarding...
[2019-05-19 19:49:46]:find connection:
[2019-05-19 19:49:46]:[PROTOCOL=UDP]:172.18.10.3:43355 -> 192.168.10.30:1234 match STOLEN:go to fast forwarding...
[2019-05-19 19:49:46]:find connection:
[2019-05-19 19:49:46]:[PROTOCOL=UDP]:172.18.10.3:43355 -> 192.168.10.30:1234 match STOLEN:go to fast forwarding...
[2019-05-19 19:49:46]:find connection:
[2019-05-19 19:49:46]:[PROTOCOL=UDP]:172.18.10.3:43355 -> 192.168.10.30:1234 match STOLEN:go to fast forwarding...
[2019-05-19 19:49:47]:find connection:
[2019-05-19 19:49:47]:[PROTOCOL=UDP]:172.18.10.3:43355 -> 192.168.10.30:1234 match STOLEN:go to fast forwarding...
```

图 5-7 过滤 UDP 前日志记录结果

配置 UDP 过滤规则后, 客户端与服务器间无法进行 UDP 通信, 如图 5-8 所示, 日志记录结果如图 5-9 所示。

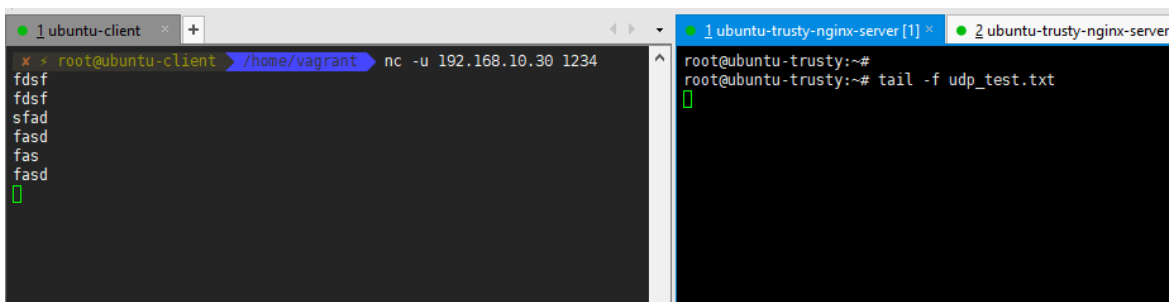


图 5-8 过滤 UDP 后内外网无法通信

```
find rule,action = 0
[2019-05-19 19:53:35]:[PROTOCOL=UDP]:172.18.10.3:60125 -> 192.168.10.30:1234 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:53:36]:[PROTOCOL=UDP]:172.18.10.3:60125 -> 192.168.10.30:1234 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:53:36]:[PROTOCOL=UDP]:172.18.10.3:60125 -> 192.168.10.30:1234 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:53:37]:[PROTOCOL=UDP]:172.18.10.3:60125 -> 192.168.10.30:1234 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:53:37]:[PROTOCOL=UDP]:172.18.10.3:60125 -> 192.168.10.30:1234 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:53:37]:[PROTOCOL=UDP]:172.18.10.3:60125 -> 192.168.10.30:1234 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:53:38]:[PROTOCOL=UDP]:172.18.10.3:60125 -> 192.168.10.30:1234 match DROP, packet dropped...
```

图 5-9 过滤 UDP 后日志记录结果

配置了 UDP 过滤规则后, 防火墙丢弃了从 172.18.10.3 的任意端口发往 192.168.10.0/24 网段的 1024~2000 端口的 UDP 数据包。

### (3) ICMP 报文过滤

配置 ICMP 过滤规则前，外网 172.18.10.3 可以 ping 通内网网关 192.168.10.254 和内网 192.168.10.30，如图 5-10 所示。

```
> root@ubuntu-client /home/vagrant ping 192.168.10.254
PING 192.168.10.254 (192.168.10.254) 56(84) bytes of data.
64 bytes from 192.168.10.254: icmp_seq=1 ttl=64 time=0.186 ms
64 bytes from 192.168.10.254: icmp_seq=2 ttl=64 time=0.188 ms
64 bytes from 192.168.10.254: icmp_seq=3 ttl=64 time=0.226 ms
64 bytes from 192.168.10.254: icmp_seq=4 ttl=64 time=0.171 ms
64 bytes from 192.168.10.254: icmp_seq=5 ttl=64 time=0.183 ms
64 bytes from 192.168.10.254: icmp_seq=6 ttl=64 time=0.177 ms
64 bytes from 192.168.10.254: icmp_seq=7 ttl=64 time=0.180 ms
^C
--- 192.168.10.254 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6000ms
rtt min/avg/max/mdev = 0.171/0.187/0.226/0.019 ms
> root@ubuntu-client /home/vagrant ping 192.168.10.30
PING 192.168.10.30 (192.168.10.30) 56(84) bytes of data.
64 bytes from 192.168.10.30: icmp_seq=1 ttl=64 time=0.371 ms
64 bytes from 192.168.10.30: icmp_seq=2 ttl=64 time=0.415 ms
64 bytes from 192.168.10.30: icmp_seq=3 ttl=64 time=0.391 ms
64 bytes from 192.168.10.30: icmp_seq=4 ttl=64 time=0.355 ms
64 bytes from 192.168.10.30: icmp_seq=5 ttl=64 time=0.429 ms
64 bytes from 192.168.10.30: icmp_seq=6 ttl=64 time=0.441 ms
^C
--- 192.168.10.30 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 4997ms
rtt min/avg/max/mdev = 0.355/0.400/0.441/0.034 ms
```

图 5-10 过滤 ICMP 前正常通信

配置 ICMP 过滤规则后，172.18.10.3 无法 ping 通 192.168.10.254 和 192.168.10.30，如图 5-11 所示，日志记录结果如图 5-12 所示。

```
* > root@ubuntu-client /home/vagrant ping 192.168.10.30
PING 192.168.10.30 (192.168.10.30) 56(84) bytes of data.
^C
--- 192.168.10.30 ping statistics ---
12 packets transmitted, 0 received, 100% packet loss, time 11080ms

* > root@ubuntu-client /home/vagrant ping 192.168.10.254
PING 192.168.10.254 (192.168.10.254) 56(84) bytes of data.
^C
--- 192.168.10.254 ping statistics ---
15 packets transmitted, 0 received, 100% packet loss, time 14103ms

* > root@ubuntu-client /home/vagrant
```

图 5-11 过滤 ICMP 后不能 ping 通

```
find rule,action = 0
[2019-05-19 19:57:35]:[PROTOCOL=ICMP]:172.18.10.3 -> 192.168.10.30 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:57:36]:[PROTOCOL=ICMP]:172.18.10.3 -> 192.168.10.30 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:57:37]:[PROTOCOL=ICMP]:172.18.10.3 -> 192.168.10.30 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:57:38]:[PROTOCOL=ICMP]:172.18.10.3 -> 192.168.10.30 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:57:42]:[PROTOCOL=ICMP]:172.18.10.3 -> 192.168.10.254 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:57:43]:[PROTOCOL=ICMP]:172.18.10.3 -> 192.168.10.254 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:57:44]:[PROTOCOL=ICMP]:172.18.10.3 -> 192.168.10.254 match DROP, packet dropped...
find rule,action = 0
[2019-05-19 19:57:45]:[PROTOCOL=ICMP]:172.18.10.3 -> 192.168.10.254 match DROP, packet dropped...
```

配置 ICMP 过滤规则后，防火墙丢弃了从 172.18.10.3 发往 192.168.10.0/24 网段的 ICMP 数据包。

清空规则后,对内外网双向通信的数据包的流向进行日志记录,如图 5-13 所示。

图 5-13 数据流快转

出的 HTTP 请求也有相同的处理结果。

5.1.4 测试结果分析

根据上述数据包过滤测试结果，本防火墙系统在添加相应的规则之前，外网的客户端与受保护网络可以进行正常通信，在加了 icmp、tcp、udp 拒绝规则之后，客户端 172.18.10.3 的发送的相应的 icmp、tcp、udp 数据包被防火墙拦截，服务器无法收到相应的请求，客户端得不到服务器返回的结果。证明防火墙包过滤功能正常生效。快转测试结果显示，被五元组标识的每一条数据流，在规则允许的情况下，第一个数据包会发往协议栈，因此，日志显示匹配到的是 f-stack 的 freebsd 协议栈，后续该流的数据包将会根据第一个包流出的网卡端口，进行快速转发，因此，日志显示匹配到的是 STOLEN。达到了快转模块的设计要求。

5.2 性能测试

5.2.1 测试环境

(1) 系统配置

本系统的性能测试使用一台 Linux 服务器，具体配置如表 5-2 所示，业务网卡使用 2 个 Intel x710 2 端口万兆网卡，支持多队列。测试时，分别将两个网卡的 0 端口（或 1 端口）与万兆发包测试仪相连进行发包测试。服务器 CPU 是 Intel E5 2620 v3，主频为 2.4Ghz，有 6 个 CPU 核心，可以绑定 6 个线程以进行数据并行处理。操作系统为 Centos7.6，内核版本为 4.4.15。服务器具体配置如表 5-2 所示

表 5-2 服务器配置

| 配置信息 | 规格                                        |
|------|-------------------------------------------|
| CPU  | Intel(R) Xeon(R) CPU E5-2620 v3 @ 2.40GHz |
| 内存   | DDR4 1600 32G                             |
| 操作系统 | CentOS Linux release 7.6.1810             |

|            |                               |
|------------|-------------------------------|
| 网卡         | 2 个 Intel X710 DA2 10GbE 光口网卡 |
| 网卡驱动       | i40e, igb_uio                 |
| DPDK 版本    | DPDK 17.11.2 LTS              |
| f-stack 版本 | F-Stack v1.12                 |
| 编译器        | gcc version 4.8.5 20150623    |

在硬件上，调整 BIOS 设置，禁用所有省电选项，选择 Performance 作为性能策略；将 CPU 频率改为 2.4Ghz（最大频率），并禁用 Turbo Boost；关闭系统超线程功能（VT-d）。确保将两张万兆网卡插入 PCIE 3.0 x8 的插槽上。

在软件上，更改 Linux 引导选项。通过 grub 文件配置，设置系统加载时保留 8 个 1G 大小的页面，系统加载时保留大页内存可以有效避免内存碎片化。为了减少 CPU 资源竞争，通过 isolcpus 参数隔离用于 DPDK 的 CPU。

## (2) F-stack 环境配置

本文测试使用的 f-stack 版本是 v1.12，f-stack 内置 DPDK 源码，版本为 17.11.2 LTS，首先到官网下载源码，进行编译。为在测试时要进行如下步骤：

- 1) 首先加载 UIO 模块，然后插入 DPDK 的 IGB\_UIO 模块
- 2) 挂载大页内存

```
mount -t hugetlbfs nodev /mnt/huge
```

- 3) 关闭 ASLR;

```
echo 0 > /proc/sys/kernel/randomize_va_space
```

- 4) 将两张光口网卡的业务端口绑定到 DPDK 的 IGB\_UIO 驱动

5) 程序运行的配置文件中 freebsd.boot 属性下加入 net.inet.ip.forwarding=1 开启 ip 转发功能。

## 5.2.2 测试方案

发包工具采用思博伦公司的万兆流量测试仪，客户端版本为 Spirent TestCenter

3.7。测试仪与服务器连接如图 5-14 所示。

本论文设计的基于 DPDK 防火墙系统是多核多线程架构，为了测试系统最大性能，流量测试仪配置多条流进行测试，服务器上以 6 核心 6 线程运行测试程序，利用 RSS 将不同的数据流通过多个队列分发到不同的 CPU 核心上，达到负载均衡的效果。在测试仪上分别设置数据包长为 64、128、256、512、1024、1280、1500，进行 RFC 2544 吞吐量测试。

测试包括四个部分：数据包捕获测试、f-stack 三层转发性能、系统快转性能、系统整体性能，在数据包捕获测试中将基于 Linux 系统内核的三层转发性能与基于 DPDK 的三层转发性能进行对比，并分析结果产生原因，在 f-stack 三层转发测试中与 linux 内核的三层转发方式进行对比，分析结果产生的原因。

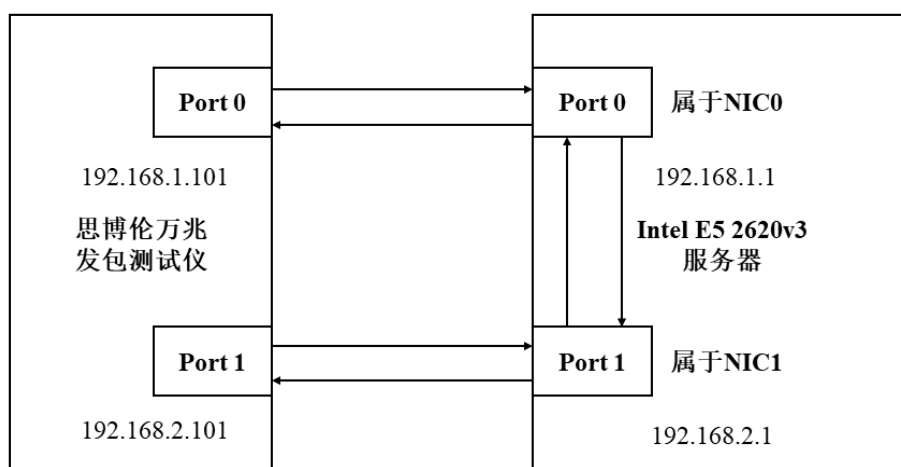


图 5-14 测试仪与服务器连接示意图

思博伦流量测试仪与服务器通过两对光纤连接，测试仪通过 0 端口发送的数据包经过服务器回到 1 端口，流量测试仪根据 0、1 端口收发数据包的数量进行统计，得出程序的吞吐量，并显示在测试仪客户端程序上。

为了测试系统的极限性能，使用全双工模式进行数据包收发。流量测试如图 5-15 所示。



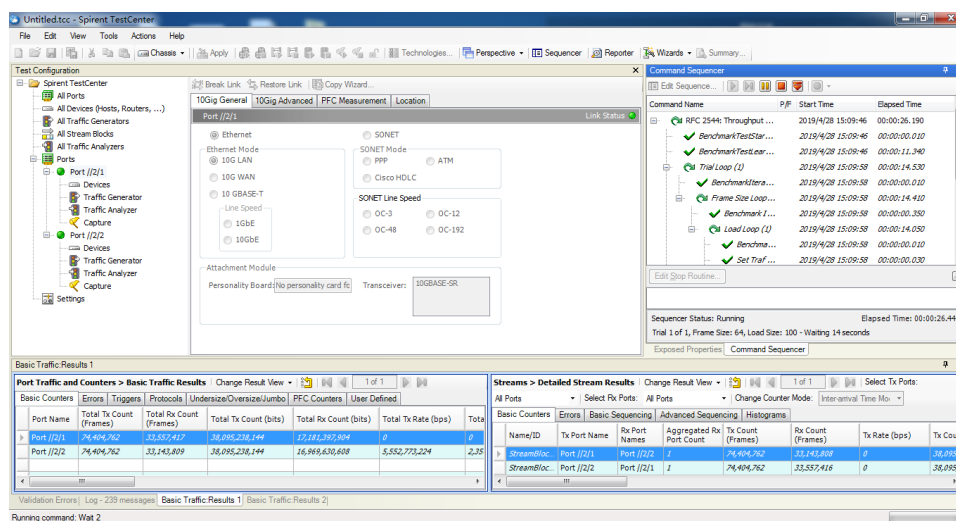


图 5-15 思博伦万兆测试仪发包过程

## 5.2.3 数据包捕获性能测试

传统的数据包捕获方式通常采用旁路操作系统内核的机制，数据包的捕获依赖操作系统网络协议栈，无法适应高速网络环境。本节将对基于 Linux 系统内核的三层转发性能与基于 DPDK 的三层转发性能进行对比，转发是指数据包从一个端口流入系统并根据路由匹配选择另一个端口将数据包发出，当路由表条目极少时，转发性能主要由数据包捕获性能决定。因此，可以通过转发性能测试，分析出报文捕获机制的优劣。测试结果如图 5-16 所示。

由中可以看出，在包长为 64 字节的情况下 Linux 三层转发吞吐率约为 2.7%，出现了极大量的丢包现象，而 DPDK 两核心两线程转发吞吐率达到了 99.9%，只丢失了 0.1%的数据包，随后，基于 DPDK 的数据包捕获方式，在数据包长为 256~1500 的各种情形下，数据包捕获率都达到了转发线速。而 Linux 三层转发，在包长为 128~512，吞吐率从 4.9%增加到 14.9%，性能有所增加，但远低于 DPDK 双核心性能，在包长为 512~1500 时，吐率从 14.9%增加到 41.8%。

经过三层转发性能测试，可以看出，DPDK 三层转发性能远远高于 Linux 转发性能。因此，可以得知，基于 DPDK 的高性能数据包捕获方式相比于传统的 Linux 内



核的数据包捕获方式，性能得到了极大的提升。

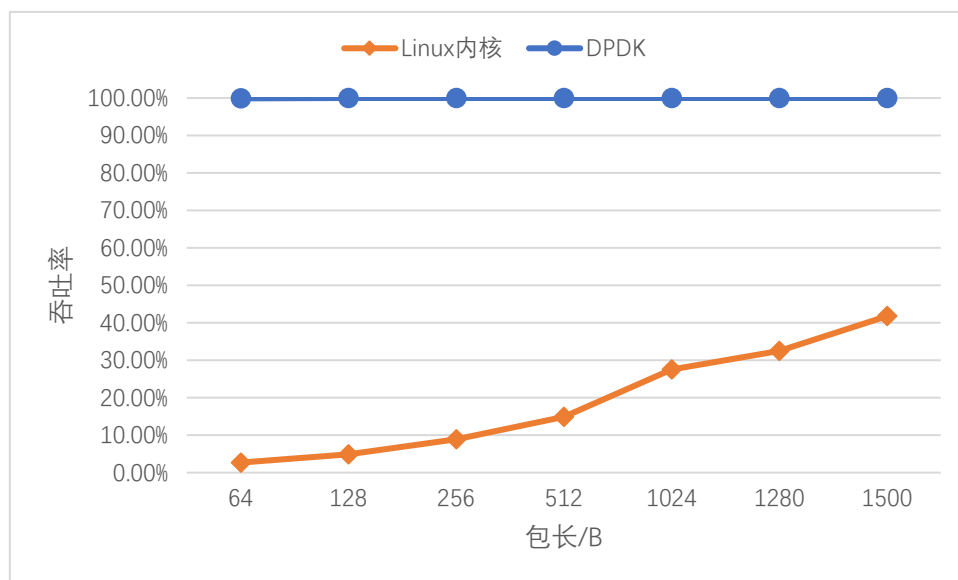


图 5-16 Linux 内核三层与 DPDK 三层转发性能对比

究其原因，Linux 三层转发依赖于操作系统内核协议栈，在数据包捕获过程中，存在两次数据包内存拷贝、进程上下文切换以及软硬件中断，消耗了大量的系统资源。而 DPDK 采用零拷贝技术与轮询模式收包、RSS 等技术，减少了数据包内存拷贝、软硬件中断和系统调用，单个数据包处理时延极大的缩短，因而能以线速捕获各种包长的数据包。

## 5.2.4 f-stack 三层转发性能测试

f-stack 三层转发性能测试主要为了测试基于 f-stack 的用户态协议栈的性能，并与传统的 linux 内核三层转发进行对比和分析。

如图 5-14 所示，在配置文件中分别设置两张网卡的 IP 地址，分别为 192.168.1.1 和 192.168.2.1，启动 f-stack 程序。

思博伦流量测试仪上配置与直连网卡端口同端的 IP 地址，连接示意图如图 5-14 所示，并设置多条数据流进行三层转发性能测试，测试仪多条流配置如图 5-17 所示。

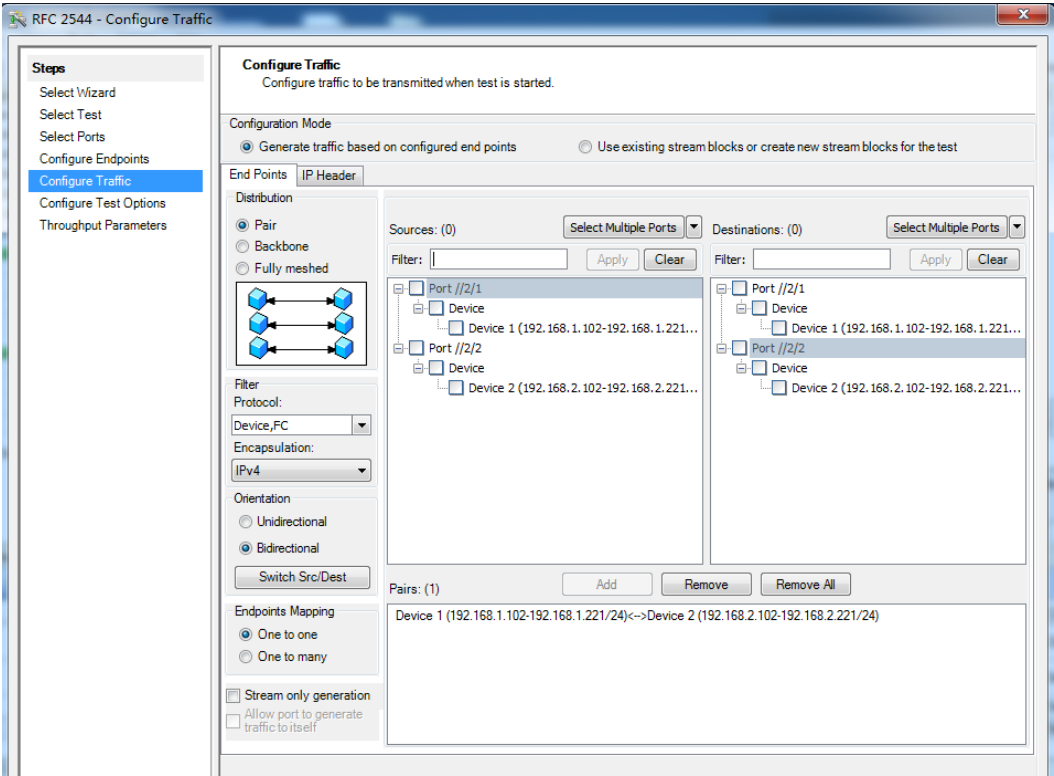


图 5-17 测试仪多条流配置

F-stack 三层转发测试结果如图 5-18 所示。

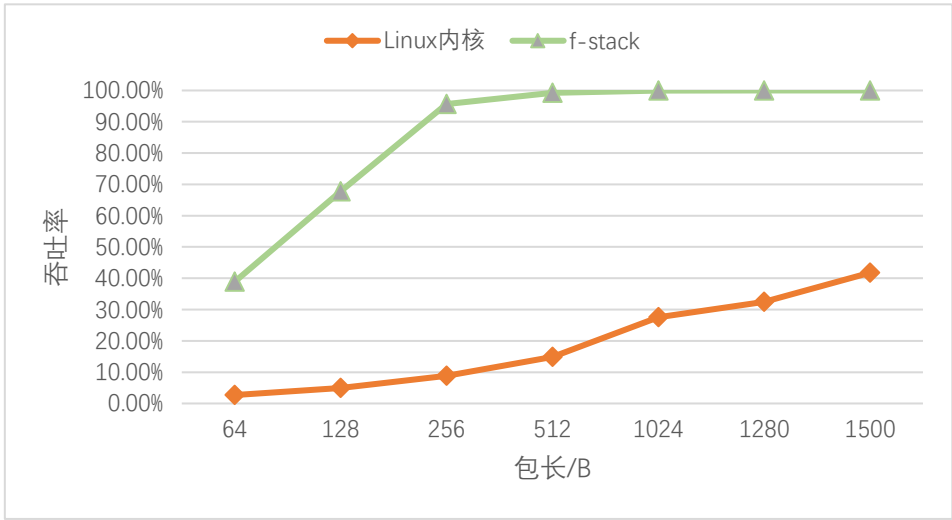


图 5-18 三层转发性能测试结果

由图 5-18 中的测试结果可知，在数据包长为 64 字节时，f-stack 用户态协议栈丢失了较多的数据包，吞吐率只有 39%左右，在 128~512 字节的情况下，随着包长

的增加, f-stack 丢包越来越少, 吞吐率从 67.78% 增加到 99.23%, 转发性能已基本达到线速。Linux 三层性能如 5.2.3 节所述, 虽然 f-stack 性能相比 DPDK 的转发性能有所下降, 但相比 Linux 三层转发性能, 性能得到较大提升, 但小包 64-128 字节时, 丢包较多, 吞吐率存在优化空间。

究其原因, f-stack 增加了用户态协议栈, 数据包在经过协议栈时需要经过冗长的处理路径, 因此性能有所降低, 但相比于传统的 Linux 内核协议栈, f-stack 的性能在 64~512 字节的情况均远高于 Linux 内核。分析其原因, f-stack 采用全核心的方式轮询地收取数据包, 减少了中断次数与系统调用; 且 f-stack 是用户态协议栈, 数据包拷贝次数相比基于 Linux 内核的转发方式减少, 因此性能有较大的提升, 但小包处理性能有待提高。

## 5.2.5 快转性能测试

开启防火墙会话查询与快转, 关闭规则查询功能, 允许防火墙为每一条数据流建立会话, 对流经系统的数据包一律放行。

按照 5.2.4 节的测试方式设置系统环境和流量测试仪。结果如图 5-19 所示, 测试结果分析将在 5.2.7 节进行。

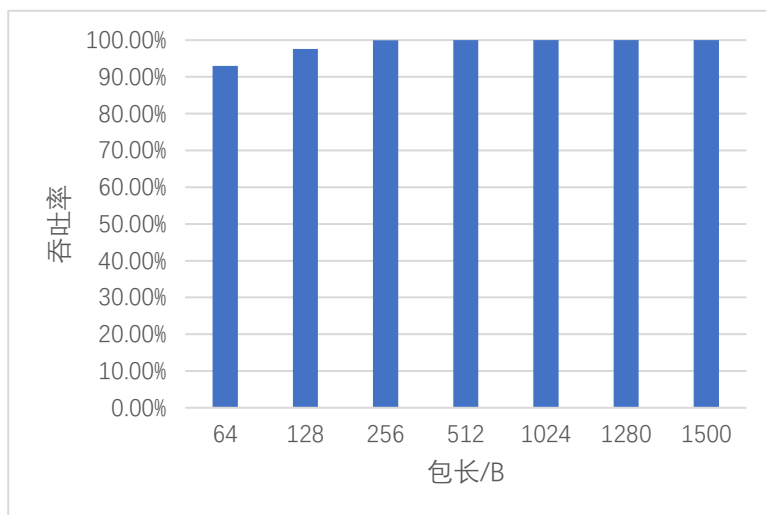


图 5-19 防火墙开启快转性能

## 5.2.6 防火墙整体性能测试

开启防火墙全部功能，包括规则查询，允许防火墙为每一条数据流建立会话，对流经系统的数据包一律放行。按照 5.2.4 节的测试方式设置 f-stack 环境和流量测试仪，测试结果如图 5-20 所示，测试结果将在 5.2.7 节进行统一分析。

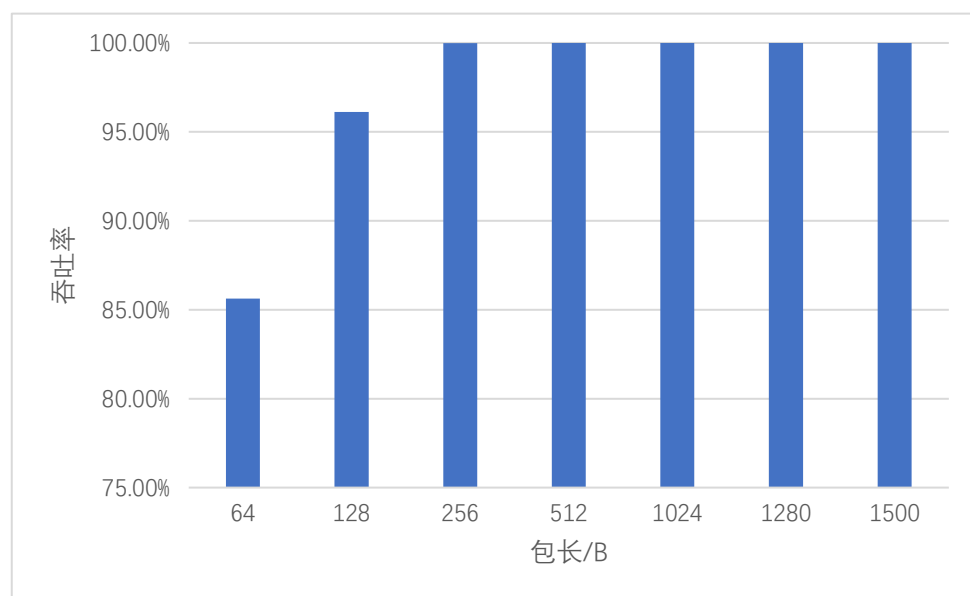


图 5-20 系统整体性能

## 5.2.7 测试结果分析

系统测试结果如图 5-21 所示，防火墙增加了快转之后，小包转发性能得到了较大的提升，包长为 64 字时，防火墙性能由 f-stack 的 38.9% 增加到 90% 以上，性能得到 1 倍以上的增长，但防火墙开启了规则查询之后，系统性能有所降低，吞吐率降到 85.6%，但相比于 f-stack，性能提高了 120%。随着包长的增加，仅开启快转的防火墙在 256 字节时丢包率不到 3%，随后在 256 字节时基本达到转发线速。而开启全部功能的防火墙在 64~256 字节，性能逐渐增加，且始终高于 f-stack，但低于仅开启快转的性能，并在 256 字节时基本达到线速。

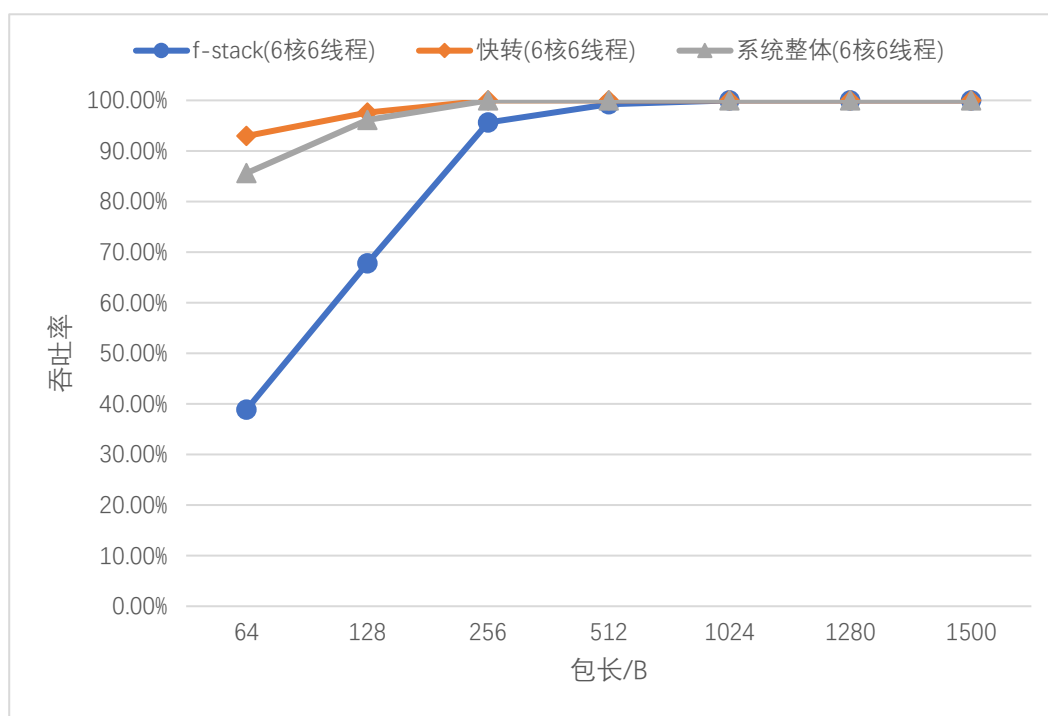


图 5-21 f-stack、防火墙开启快转、防火墙整体性能对比

分析其原因，快转性能较高的原因在于，系统在建立会话之后，会查询会话表，并根据会话表中流的快转标志进行数据包快速转发，因而绕过了冗长的协议栈处理路径。但开启全部功能的防火墙由于需要在建立会话之前，进行规则查询，因此性能相比于仅开启快转的方式略有下降。

## 5.3 本章小结

本章首先对系统功能进行了测试，介绍了功能测试的环境配置和网络拓扑，详细阐述了系统包过滤功能和快转功能的测试过程以及测试结果，并对测试结果进行分析，证明了本系统包过滤功能和快转功能正常生效。然后对系统性能进行测试，给出了测试服务器的配置、测试前针对软硬件的设置以及 f-stack 环境配置；进而阐述了系统测试方案，按照系统测试方案，将测试分为四个部分进行，最后对测试结果进行了对比分析。

## 6 总结与展望

本章第一节对全文内容进行总结，简要阐述了基于 DPDK 的高性能防火墙系统的主要研究成果，通过设计并实施测试方案，与相应指标进行对比，可以证明本系统与传统 Linux 防火墙相比，性能有大幅度的提升。第二节对本系统进行整体评估，指出有待改进的地方，为下一步的研究奠定基础，并提供思路 and 方向。

### 6.1 全文总结

防火墙是系统的第一道屏障，对保障系统安全有非常重要的意义。然而随着光纤技术的发展，网络带宽快速提升，硬件性能极速提高，但软件技术相对落后，传统的网络协议栈对数据包的处理流程复杂，开销较大，基于传统网络协议栈的防火墙已无法满足人们对于高性能处理密集数据的要求，因此研究开发高性能防火墙具有十分重要的意义。本文针对大流量的高速网络环境下传统防火墙数据包处理效率低的问题，提出了一种基于 DPDK 的高性能防火墙系统，旨在为处理密集数据提出解决方案，主要工作成果和创新点如下：

(1) 通过分析网络设备发展的现状，了解基于传统协议栈的防火墙性能不足的原因，表明研究和开发高性能防火墙系统的重要性。然后从包过滤性能优化和网络数据包处理性能优化这两个方面分析国内外研究现状，通过比较研究方案的优缺点，综合分析选用 DPDK 作为开发的基础框架。

(2) 结合系统实际应用场景，从功能性和非功能性两方面分析用户需求，确定系统开发方向，根据需求分析的结果对系统进行总体设计，划分系统的主要功能模块：数据包捕获模块、会话查询模块、规则查询模块、快转模块和管理模块，对各个模块进行详细设计和实现。

(3) 数据包捕获模块利用了 DPDK 提供的包捕获相关接口进行实现，遵循接收队列、线程、CPU 逻辑核、内存池相互之间 1 对 1 的资源分配原则，利用 DPDK 轮

询技术和批量读取方式接收数据包，旨在充分利用 DPDK 的高性能收包特性，提高系统接收数据包的性能。

(4) 会话查询模块分为会话信息设计、会话表维护和会话查询算法三个具体功能，根据会话表中的时间戳，对会话表定期维护，减少会话表的空间资源的浪费且利于维持会话查询效率稳定；会话查询算法使用哈希算法，用链地址法解决冲突，保证会话查询的高效性。

(5) 规则查询模块通过分析传统顺序查询方式效率较低，提出一种基于分治法的规则查询算法，根据协议类型划分子规则集，减少数据包尝试匹配的规则集数量，在子规则集中根据规则间的相关性将规则分为有序组和无序组，依据两组各自的规则特点，有序组使用顺序查询算法，无序组使用基于 Radix Trie 树的最长前缀匹配算法，提高数据包在规则集中的查询效率。

(6) 快转模块通过会话信息快速转发符合条件的数据包，避免数据包经过协议栈处理导致的性能损耗，而且降低了数据包的排队时延，进一步提高了数据包的转发性能。

(7) 当数据包无法被快速转发时，将由协议栈处理，本系统的协议栈功能由用户态网络框架 f-stack 完成，f-stack 底层基于 DPDK 开发，并采用多进程无共享架构避免资源竞争，与传统协议栈相比，f-stack 数据包处理效率有较大提升。

综上所述，本文提出的基于 DPDK 的高性能防火墙系统通过底层使用 DPDK 接收数据包，并对防火墙各模块的传统实现方式进行优化，来提升防火墙系统的性能。通过对系统进行测试，不仅证明了本系统在性能上的优越性，也证明了本课题的研究对满足大流量环境下提升处理密集数据性能的需求有重要意义。

## 6.2 课题展望

本文提出的基于 DPDK 的高性能防火墙系统可以提高防火墙对数据包的处理性能，但是仍存在一些不足之处。本系统重点关注性能的提升，相对于性能方面，某些

功能的实现有所忽略。

(1) 本系统支持的协议类型比较少, 仅支持一些常见协议, 例如 TCP、UDP、ICMP、IP 等, 导致不能对所有数据包精准过滤, 在以后的研究过程中, 还需要增加对其他更多协议类型的支持。

(2) 本系统尚未实现策略路由, 因此无法按照用户指定的策略进行路由选择, 不能很好地提供精细化管理服务, 后续的研究工作还需将基于源/目的 IP 地址的路由选择方式与实际应用中的多种因素进行结合, 实现策略路由的功能。

(3) 本系统以单台服务器的形式, 在高速网络环境中对流量进行负载均衡的方式是利用网卡多队列技术, 通过 RSS 将流量分配到指定队列上, 并采用接收队列、线程/进程、CPU 逻辑核、内存池两两之间一对一的方式分配资源。这样的流量分配方式会随着流量的不断增加导致资源瓶颈。因此后续研发中需要调整系统架构为分布式, 这样会降低单台服务器的压力, 解决资源瓶颈的问题。



# 华中科技大学硕士学位论文

---

## 致谢

两年转眼间就过去了，硕士阶段的这两年我学到了很多也成长了很多。这两年的结束对我来说也意味着新的挑战的开始。

首先我要感谢母校——华中科技大学，感谢学校对我的栽培，让我增长了许多知识和见识。大学的环境就是一个小型的社会，学校让我更加懂得如何为人处世，如何融入社会，这是书本上学不到的宝贵经验。

其次我要感谢我的指导导师梅松，梅老师对待硕士研究生毕业设计非常用心，在系统的设计阶段，让我定期汇报工作，给我提出修改意见，督促我尽快完成设计；在论文撰写阶段，老师让我定期给他汇报完成情况，在了解论文进度的同时还仔细看了论文内容，给我提了不少修改的建议。在他的悉心教导和教诲下，我的毕业设计和论文才能顺利完成。梅老师对人对事认真的态度和给予我的很多指导和帮助，让我受益良多。

感谢张云鹤老师、王美珍老师和肖凌老师，感谢他们给予我的帮助。我在写论文的过程中，经常有不懂的地方会发给老师们请求指导，老师们在忙碌的教学和研究工作中还抽出不少的时间，细致地为我分析论文的不足之处并提出指导意见，为我论文的完成提供了很多宝贵的建议，让我更好地完成了毕业论文。

感谢同实验室的几位同学在生活和学习上对我的帮助，在进行系统实现时帮助我调优测试，在撰写论文时为我提出行文建议。

感谢我的父母抚养了我这么多年，教育我成材，教我做人的道理。如今我将硕士研究生毕业，踏入社会努力工作。

最后，衷心地感谢所有读到此文的老师和同学。我将在以后的工作中继续努力，以成果回报大家的期待。

## 参考文献

- [1] 杨佑君. 基于 DPDK 的高性能 DDoS 攻击防御系统设计与实现: [硕士学位论文]. 北京: 北京交通大学, 2018
- [2] 吴延卯. iptables 防火墙性能优化研究. 计算机与现代化, 2012, (09):106-108
- [3] Mccanne S, Jacobson V. The BSD Packet Filter: A New Architecture for User-Level Packet Capture. In: Proceedings of the USENIX winter, 1993
- [4] 穆瑞超. 基于 DPDK 的高性能 VPN 网关的研究与实现: [硕士学位论文]. 哈尔滨: 哈尔滨工业大学, 2017
- [5] 蔡鹏程. 基于 FPGA 动态包过滤防火墙系统设计: [硕士学位论文]. 南京: 东南大学, 2016
- [6] Cancelo G I, Hansen S. NETMAP: An Integrated System for Building Analog Neural Networks. Journal of systems architecture, 1997, 44(1): 63~73
- [7] 陈睿. 基于网络处理器的 PPU 系统中会话管理及流量统计模块的设计与实现: [硕士学位论文]. 北京: 北京邮电大学, 2008
- [8] 凌质亿. 面向高速网络环境的实时入侵检测系统的研究与实现: [硕士学位论文]. 南京: 东南大学, 2016
- [9] Shimonishi H, Murase T. A Network Processor Architecture for Flexible QoS Control in Very High-Speed Line Interfaces. In: Proceedings of the 2001 IEEE Workshop on High Performance Switching and Routing (IEEE Cat. No. 01TH8552). IEEE Press, 2001. 402~406
- [10] Shah N. Understanding Network Processors: [Master's thesis]. Berkeley: University of California, 2001
- [11] Murta C D, Jonack M A. Evaluating Livelock Control Mechanisms in a Gigabit Network. In: Proceedings of the 15th International Conference on Computer

- Communications and Networks. New York: IEEE Press, 2006. 40~45
- [12] Von Eicken T, Basu A, Buch V et al. U-Net: A User-Level Network Interface for Parallel and Distributed Computing. In: Proceedings of the 15th ACM Symposium on Operating Systems Principles. New York:ACM Press, 1995. 40~53
- [13] Deri L. Improving Passive Packet Capture: Beyond Device Polling. In: Proceedings of SANE. 2004. 85~93
- [14] Kumar S , Spafford E. A Pattern Matching Model For Misuse Intrusion Detection. Computers & Security, 2010, (1):28
- [15] 韩德志, 谢长生. 一种高性能防火墙系统的设计与实现. 计算机应用, 2000, (07):32~35
- [16] Harn L , Lin H Y . Integration of User Authentication and Access Control. Computers & Digital Techniques Iee Proceedings E, 1992, 139(2):139~143
- [17] Yen S, Lai H C. Analysis and Improvement of an Access Control Scheme with User Authentication. IEE Proceedings-Computers and Digital Techniques, 1994, 141(5): 271~273
- [18] Ros-Giralt J, Commike A, Honey D et al. High-Performance Many-Core Networking: Design and Implementation. In: Proceedings of the Second Workshop on Innovating the Network for Data-Intensive Science. New York : ACM Press, 2015. 1
- [19] Rajesh R, Ramia K B, Kulkarni M. Integration of LwIP Stack Over Intel (R) DPDK for High Throughput Packet Delivery to Applications. In: Proceedings of the 2014 Fifth International Symposium on Electronic System Design. IEEE Press, 2014. 130~134
- [20] 王佰玲, 方滨兴, 云晓春. 零拷贝报文捕获平台的研究与实现. 计算机学报, 2005, 28(01): 46~52
- [21] Asai H. Where are the Bottlenecks in Software Packet Processing and Forwarding?:

- Towards High-Performance Network Operating Systems. In: Proceedings of The Ninth International Conference on Future Internet Technologies. New York: ACM Press, 2014. 5
- [22] Liao G, Znu X, Bnuyan L. A New Server I/O Architecture for High Speed Networks. In: Proceedings of The 2011 IEEE 17th International Symposium on High Performance Computer Architecture. IEEE Press, 2011. 255~265
- [23] Gallenmüller S, Emmerich P, Wohlfart F et al. Comparison of Frameworks for High-Performance Packet IO. In: Proceedings of the Eleventh ACM/IEEE Symposium on Architectures for networking and communications systems. IEEE Press, 2015. 29~38
- [24] Rizzo L. Netmap: A Novel Framework for Fast Packet I/O. In: Proceedings of the 21st USENIX Security Symposium (USENIX Security 12). 2012. 101~112
- [25] Du J, Liu P. Design and Implementation of Efficient One-Way Isolation System Based on PF\_RING. In: Proceedings of the 2012 Fourth International Conference on Multimedia Information Networking and Security. IEEE Press, 2012. 105~108
- [26] Ajayan A C, Prabakaran P, Krishnan M R et al. Hiper-Ping: Data Plane Based High Performance Packet Generation Bypassing Kernel On x86 Based Commodity Systems. In: Proceedings of the 2016 International Conference on Advances in Computing, Communications and Informatics (ICACCI). IEEE Press, 2016. 478~483
- [27] Xu Z, Liang Z, Li F et al. PacketUsher: A DPDK-based Packet I/O Engine for Commodity PC. In: Proceedings of the IEEE/CIC International Conference on Communications in China. IEEE Press, 2016. 1~5
- [28] Yurchenko M, Cody P, Coplan A et al. OpenNetVM: A Platform for High Performance NFV Service Chains. In: Proceedings of the Symposium on SDN Research. New York:ACM Press, 2016. 26~31
- [29] 何佳伟, 江舟. 基于 Intel DPDK 的高性能网络安全审计方案设计. 电子测试,

2016, Z1(038): 87~91

- [30] Redžović H, Vesović M, Smiljanić A et al. Energy-Efficient Network Processing Based On Netmap Framework. *Electron Lett*, 2017, 53(6): 407~409
- [31] Kim J, Na J. A Study On One-Way Communication Using PF\_RING ZC. In: *Proceedings of the 2017 19th International Conference on Advanced Communication Technology (ICACT)*. IEEE Press, 2017. 301~304
- [32] Han S, Jang K, Park K S, et al. PacketShader: a GPU-accelerated software router. *ACM SIGCOMM Computer Communication Review*, 2011, 41(4): 195~206
- [33] Honda M, Huici F, Raiciu C, et al. Rekindling network protocol innovation with user-level stacks. *ACM SIGCOMM Computer Communication Review*, 2014, 44(2): 52~58
- [34] Tang L, Yan J, Sun Z et al. Towards High-Performance Packet Processing On Commodity Multi-Cores: Current Issues and Future Directions. *Science China Information Sciences*, 2015, 58(12): 1~16
- [35] Li B, Tan K, Luo L L et al. Clicknp: Highly Flexible and High Performance Network Processing with Reconfigurable Hardware. In: *Proceedings of the 2016 ACM SIGCOMM Conference*. New York:ACM Press, 2016. 1~14
- [36] Shim J, Kim J, Lee K et al. Knapp: A Packet Processing Framework for Manycore Accelerators. In: *Proceedings of the 2017 IEEE 3rd International Workshop on High-Performance Interconnection Networks in the Exascale and Big-Data Era (HiPINEB)*. IEEE Press, 2017. 57~64
- [37] 吴承. 用户态 IPSec 协议栈的研究与实现: [硕士学位论文]. 西安: 西安电子科技大学, 2014
- [38] 刘寿强, 陈梓忠, 陈晓霞. Linux 个人防火墙设计与实现——数据包处理部分. *计算机安全*, 2006, (01): 17~20
- [39] 任昊哲, 年梅. 基于 DPDK 的高速数据包捕获方法. *计算机系统应用*, 2018,

27(06): 240~243

- [40] X. Zhang, Y. Jiang and M. Cong, Performance Improvement for Multicore Processors Using Variable Page Technologies. In: Proceedings of the 2011 IEEE Sixth International Conference on Networking, Architecture, and Storage. IEEE Press, 2011. 30~235
- [41] Fusco F, Deri L. High Speed Network Traffic Analysis with Commodity Multi-Core Systems. In: Proceedings of the 10th ACM SIGCOMM conference on Internet measurement. New York: ACM Press, 2010. 218~224
- [42] Pongrácz G, Molnár L, Kis Z L. Removing Roadblocks From SDN: OpenFlow Software Switch Performance On Intel DPDK. In: Proceedings of the 2013 Second European Workshop on Software Defined Networks. IEEE Press, 2013. 62~67